Future Generation Computer Systems 183 (2026) 108586

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

From XAI to MLOps: Explainable concept drift detection with the partial dependence profiles-based approach ☆

Ugur Dar, Mustafa Cavus *

Eskisehir Technical University, Department of Statistics, Eskisehir, 26555, Turkiye

ARTICLE INFO

Dataset link: https://github.com/ugurdar/xai-mlops

Keywords:
Concept drift
Model monitoring
MLOps
Explainable AI
Explainable drift detection

ABSTRACT

Predictive models often degrade in performance due to evolving data distributions, a phenomenon known as data drift. Among its forms, concept drift — where the relationship between explanatory variables and the response variable changes — is particularly challenging to detect and adapt to. Traditional drift detection methods often rely on metrics such as accuracy or marginal variable distributions, which may fail to capture subtle but important conceptual changes. This paper proposes a novel method, *Profile Drift Detection* (PDD), which enables both the detection of concept drift and an enhanced understanding of its underlying causes by leveraging an explainable AI tool—Partial Dependence Profiles (PDPs). PDD quantifies changes in PDPs through new drift metrics that are sensitive to shifts in the data stream while remaining computationally efficient. This approach is aligned with MLOps practices, emphasizing continuous model monitoring and adaptive retraining in dynamic environments. Experiments on synthetic and real-world datasets demonstrate that PDD outperforms existing methods by maintaining high predictive performance while effectively balancing sensitivity and stability in drift signals. The results highlight its suitability for real-time applications, and the paper concludes by discussing the method's advantages, limitations, and potential extensions to broader use cases.

1. Introduction

The stationary and unbiased distribution assumption in predictive models states that observations in the training and test sets are independent and identically distributed [1]. However, this assumption often does not hold in practice, leading to significant decreases in the performance of predictive models [2]. Data characteristics can change over time, rendering a trained model obsolete and forcing it to adapt to these changes. The constantly evolving nature of data streams presents new challenges for predictive models [3]. These models must demonstrate high predictive accuracy and quickly incorporate new information while remaining computationally light and responsible [4]. Model monitoring, which is a part of MLOps, has become a necessary additional layer in the predictive modeling process in Fig. 1 to ensure that the ML model maintains consistent behavior over time in real-world applications [5,6].

One of the problems that causes inconsistency in behavior is called data drift, whose types are also classified into three categories: (1) Covariate drift, (2) Label drift, and (3) Concept drift. Data drift may manifest in various forms, such as increased electricity consumption due to significant life changes like pandemic-related lockdowns [7], changes in airline customer behaviors [8], and shifts in supply chain planning due to evolving consumer behavior [9]. Beyond tabular data, drift detection has become crucial in specialized domains, such as identifying AI-generated content through multimodal feature fusion [10] and developing interpretable frameworks for intrusion detection in industrial Internet of Things ecosystems [11]. It is essential to detect these drifts on time to take precautions for the model.

Various methods are used to detect data drifts, categorized into three groups: statistical, sequential analysis-based, and window-based methods. For detecting concept drift, which is characterized as a change in the relationship between explanatory variables and the response variable, there are instances where current methods in the literature fall short [12–14]. This limitation highlights the need for MLOps, which integrates model monitoring and adaptation strategies to address data drift effectively. MLOps practices focus on maintaining model performance and managing change, enabling predictive models to adapt to dynamic data environments in real time. One particularly challenging scenario is when concept drift occurs without performance and distribution metrics changes, rendering traditional detection methods unreliable. In response to this situation, some alternative approaches

☆ This article is part of a Special issue entitled: 'MLOps' published in Future Generation Computer Systems.

* Corresponding author.
E-mail address: mustafacavus@eskisehir.edu.tr (M. Cavus).

https://doi.org/10.1016/j.future.2026.108586
Received 24 February 2026; Received in revised form 4 May 2026; Accepted 6 May 2026
Available online 11 May 2026
0167-739X/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

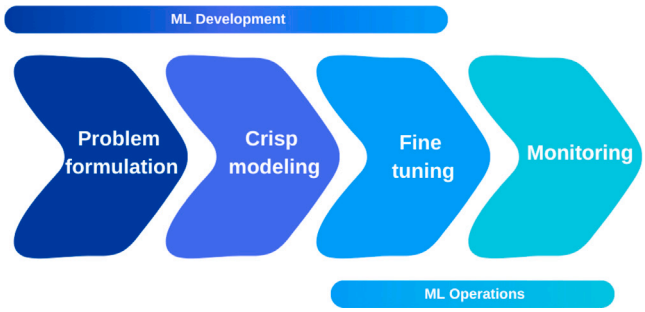


Fig. 1. The predictive modeling process.

have been proposed where the methods used in the literature fail to detect concept drift adequately.

Moreover, [12] emphasized that current data drift detection methods rely solely on performance and distribution metrics, which are insufficient to understand the underlying cause of changes in the model. They developed an alternative method using the local interpretable model-agnostic explanations [15] and demonstrated through comparative studies that it offers better detection performance than existing methods. [13] used the normalized SHAP method [16] to detect data drifts in emergency department length-of-stay prediction models following COVID-19, noting that traditional methods failed to reflect changes in the model's information utilization. [17] developed an approach to detect concept drifts using permutation-based variable importance [18] values. However, they highlighted the need to address complexity, effectiveness, sensitivity, and explainability for their approach to be effectively applied. [14] explored data drifts observed in models using the SHAP method for calculating the variable importance. Their most significant challenge was the inability to distinguish between covariate and concept drift, primarily due to the inappropriate use of XAI tools. While SHAP provides sufficient information on variable importance, it fails to detect the relationship between explanatory and response variables. [6] emphasized that statistical tests operate solely based on the data distribution and may lead to false alarms. Their study developed an explainable concept drift detection method for regression problems using non-parametric bootstrap uncertainty estimation and SHAP values. [19] highlighted a lack of studies focusing on the causes and explainability of concept drift detection methods. They developed a technique based on a decision tree model to detect and visualize concept drift by identifying changes in the regions partitioned by the decision tree. However, they discussed the limitations of their approach, such as the cost of continuously training the model and the feasibility of tracking the limited number of variables selected by the model. Additionally, when non-tree-based models are used, the method might evaluate different relationships from those in the decision process, making it more suitable as a tree-based method. [17] developed an approach for detecting concept drift using permutation-based variable importance values. Similarly, [7] developed an incremental permutation-based variable importance method for use in incremental models. However, they emphasized that for their approach to be effectively applied, it is necessary to address shortcomings in complexity, efficiency, sensitivity, and explainability.

There is a limited body of literature on data drift detection methods that employ XAI tools. Even when these methods are used, they often face challenges in some data drifts, or they rely on high computational cost methods such as permutation-based variable importance or SHAP. These approaches are not always practical for detecting changes in the relationship between the response variable and explanatory variables. Therefore, this paper introduces a solution based on the partial dependence profile (PDP), which is an XAI tool used to uncover the relationship between the response variable and explanatory variables. Because it is a versatile tool that is not only used to examine the relationship between the response variable and explanatory variables but also used in hyper-parameter optimization [20], for determining variable importance and identifying interactions between variables [21], and providing more accurate explanations through the Rashomon perspective while quantifying uncertainty [22,23].

In addition to the scientific literature, software, and technologies that include data drift detection methods, which are complex to implement, they are quite limited. Although there are basic R tools such as the vetiver framework and pins for monitoring model metrics, they do not provide specific instruments for data drift monitoring. On the other hand, drifter offers useful functions for detecting data drift based on distances between distributions, residuals, and PDPs of two models, but it is limited to comparing two trained models and is not suitable for streaming data. Meanwhile, harbinger supplies some functions to detect anomalies, drifts, and change points, but is focused on time series data. While R and CRAN are immature in terms of model monitoring, there are many technologies and Python libraries available such as EvidentlyAI,[1] SeldonIO,[2] NannyML,[3] Frouros,[4] River.[5]

To address these gaps, we propose a novel concept drift detection method, Profile Drift Detection (PDD), based on explainable AI techniques. PDD leverages Partial Dependence Profiles to monitor concept drifts in production environments. The key contributions of our study are threefold: First, we introduce PDD as an explainable drift detection method that quantifies changes in the relationship between variables and the target variable through three complementary metrics (PDI, L2, and L2Der). Second, through comprehensive experiments on synthetic and real-world datasets, we demonstrate that PDD effectively balances sensitivity and stability in drift detection while maintaining high predictive performance. Third, unlike black-box methods, PDD provides interpretable insights into which variables drive the drift and how their relationships with the target variable evolve.[6]

2. Preliminaries

Let $f_\theta : \mathcal{X} \to \mathcal{Y}$ be a predictive model, where f_θ represents the function learned by the model, θ denotes the parameters to be optimized, $\mathcal{X}$ is the input space, and $\mathcal{Y}$ is the output space. The model is trained on a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathcal{X}$ are the explanatory variables and $y \in \mathcal{Y}$ is the response variable. The learning process aims to find the optimal parameters $\hat{\theta}$ by minimizing a loss function $L(f_\theta(x_i), y_i)$, which quantifies the difference between the model's predictions $f_\theta(x_i)$ and the true values y_i. The objective is to solve $\hat{\theta} = \arg\min_\theta \sum_{i=1}^n L(f_\theta(x_i), y_i)$, thereby achieving the best possible model performance on the given data. Data drift occurs when there is a discrepancy between the distribution of the data used to train a model, denoted as $P_{\text{train}}(X, Y)$, and the distribution of the data the model encounters during deployment or testing denoted as $P_{\text{test}}(X, Y)$. This drift can negatively impact the model's performance, as the model learns patterns based on the train data distribution.

Covariate drift. It occurs when the distribution of explanatory variables changes while the distribution of the response variable remains unchanged:

$$P_{\text{train}}(X) \neq P_{\text{test}}(X). \tag{1}$$

[1] https://www.evidentlyai.com/.
[2] https://www.seldon.io/.
[3] https://nannyml.readthedocs.io/en/stable/index.html.
[4] https://frouros.readthedocs.io/en/latest/.
[5] https://riverml.xyz/0.8.0/examples/concept-drift-detection/.
[6] An R implementation of PDD and baseline methods is available in the datadriftR package (CRAN). Source code and reproducibility materials: https://github.com/ugurdar/xai-mlops.

In this case, the model might struggle to make accurate predictions on new data, even though it performed well on the train set.

Label drift. It indicates a change in the distribution of the response variable while the distribution of the explanatory variable stays constant:

$$P_{\text{train}}(Y) \neq P_{\text{test}}(Y). \tag{2}$$

Label shift is particularly problematic in scenarios with imbalanced classes, as it can increase misclassification rates.

Concept drift. It occurs when the conditional relationship between explanatory variables and the response variable changes:

$$P_{\text{train}}(Y|X) \neq P_{\text{test}}(Y|X), \tag{3}$$

even if the marginal distributions $P(X)$ and $P(Y)$ remain unchanged. This fundamentally alters how the model interprets input variables, leading to reduced accuracy and reliability.

There are several methods proposed to detect these kinds of drifts in the literature, which are described in the following section.

3. Methods

This section describes the existing drift detection methods, such as the Drift Detection Method, the Early Drift Detection Method, the Hoeffding Drift Detection Method, the Kolmogorov–Smirnov test-based Windowing, and Page–Hinkley, as well as our proposed concept drift detection method, Profile Drift Detection.

3.1. Hoeffding's Drift Detection Method on average

The HDDM method [24] includes two different approaches: mean and weighted. Both methods are based on Hoeffding's Inequality. HDDM-A is used to detect abrupt concept drifts in data streams. It monitors changes in the data stream using a simple moving average and detects significant modifications based on Hoeffding's Inequality. Similar to DDM, two different confidence levels are set for alert and concept drift detection. Confidence levels α_D are set for detecting concept drift and α_W for detecting alerts. The total number of samples is denoted by n, and the cumulative sum for each new sample is $c = \sum_{i=1}^{n} x_i$. The data stream is divided into two windows: minimum window $(n_{\min}, c_{\min})$ and maximum window $(n_{\max}, c_{\max})$. These windows are used to compare the past and current states of the data. If the minimum and maximum window values are in their initial state, they are updated (if $n_{\min} = 0$ then $n_{\min} = n$, $c_{\min} = c$; if $n_{\max} = 0$ then $n_{\max} = n$, $c_{\max} = c$). Error bounds for the minimum window and total sample size are calculated using Hoeffding's Inequality:

$$\epsilon_{\alpha_D} = \sqrt{\frac{1}{2n} \ln\left(\frac{1}{\alpha_D}\right)} \tag{4}$$

$$\epsilon_{\alpha_{D1}} = \sqrt{\frac{1}{2n_{\min}} \ln\left(\frac{1}{\alpha_D}\right)} \tag{5}$$

$$\epsilon_{\alpha_{D2}} = \sqrt{\frac{1}{2n_{\max}} \ln\left(\frac{1}{\alpha_D}\right)} \tag{6}$$

Using these bounds, the values of $n_{\min}$, $c_{\min}$, $n_{\max}$, and $c_{\max}$ are updated as follows:

$$\text{If } \frac{c_{\min}}{n_{\min}} + \epsilon_{\alpha_{D1}} \geq \frac{c}{n} + \epsilon_{\alpha_D} \text{ then } c_{\min} = c,\ n_{\min} = n \tag{7}$$

$$\text{If } \frac{c_{\max}}{n_{\max}} - \epsilon_{\alpha_{D2}} \leq \frac{c}{n} - \epsilon_{\alpha_D} \text{ then } c_{\max} = c,\ n_{\max} = n \tag{8}$$

$$\frac{c}{n} - \frac{c_{\min}}{n_{\min}} \geq \sqrt{\frac{n - n_{\min}}{n_{\min}} \frac{1}{2n} \ln\left(\frac{1}{\alpha}\right)} \tag{9}$$

$$\frac{c_{\max}}{n_{\max}} - \frac{c}{n} \geq \sqrt{\frac{n - n_{\max}}{n_{\max}} \frac{1}{2n} \ln\left(\frac{1}{\alpha}\right)} \tag{10}$$

When the conditions in Eq. (7) or (8) are met, with α replaced by α_D, a concept drift is detected, and an alert state is identified when α_W is used.

3.2. Hoeffding's Drift Detection Method on weighted

HDDM-W is an algorithm that uses Exponentially Weighted Moving Averages (EWMA) to detect concept drifts in data streams. This algorithm tracks average changes occurring in the data stream and determines whether these changes are significant using Hoeffding's Inequality. The parameter λ is used in the EWMA statistic, where smaller values give less weight to the most recent observations and λ ranges from 0 to 1. It is calculated as $\hat{\mu}_{\text{EWMA}} = \lambda_{x_i} + (1 - \lambda)\hat{\mu}_{\text{EWMA}}$. Similarly, separate α values must be set for concept drift and alert states.

While HDDM-A is generally more effective in detecting abrupt concept drifts, HDDM-W performs better in detecting gradual concept drifts. HDDM algorithms do not make any distribution assumptions for the observations being compared. Variables can take values in the range $[a, b]$, and Hoeffding's Inequality can be used. However, in practice, Python modules such as scikit-multiflow and river, similar to DDM and EDDM, attempt to detect concept drift based on whether the response variable is correctly predicted.

3.3. Kolmogorov–Smirnov windowing method

KSWIN is a non-parametric statistical method used to detect concept drifts in data streams [25]. This method utilizes the Kolmogorov–Smirnov test (KS test) to identify statistical differences between new and historical data. It operates using a sliding window (Ψ) that continuously holds the most recent n data points. Two sub-windows are created from Ψ:

R, which contains the most recent r observations:

$$R = \{x_i \in \Psi \mid i > n - r\} \tag{11}$$

W, which contains r observations randomly chosen from the older part of the window:

$$W = \{x_i \in \Psi \mid i \leq n - r \text{ and } P(x) = \text{UNIF}(x_i \mid 1, n - r)\} \tag{12}$$

where $|W| = |R| = r$. The Kolmogorov–Smirnov (KS) test is then applied to compare the empirical cumulative distributions of R and W. The test calculates the maximum absolute difference between these distributions $(dist_{w,r})$:

$$dist_{w,r} = \sup_x \left|F_W(x) - F_R(x)\right| \tag{13}$$

If this distance exceeds the critical value (D_α), it indicates a concept drift:

$$dist_{w,r} > c(\alpha)\sqrt{\frac{n+r}{nr}} = \sqrt{-\frac{1}{2}\ln\alpha}\sqrt{\frac{n+r}{nr}} = \sqrt{-\frac{\ln\alpha}{r}} \tag{14}$$

When the distribution $P(X)$ changes over time while $P(y \mid X)$ remains the same, this indicates a virtual concept drift. The KSWIN test is used to detect concept drift under the assumption that any change in $P(X)$ will also result in a change in $P(y \mid X)$. Since the KSWIN test is non-parametric, there is no need to assume any specific distribution for the monitored variable. The KSWIN test can be applied to either the response variable or explanatory variables and is used to detect data drift even before actual values are observed, by using explanatory variables to predict the response variable.

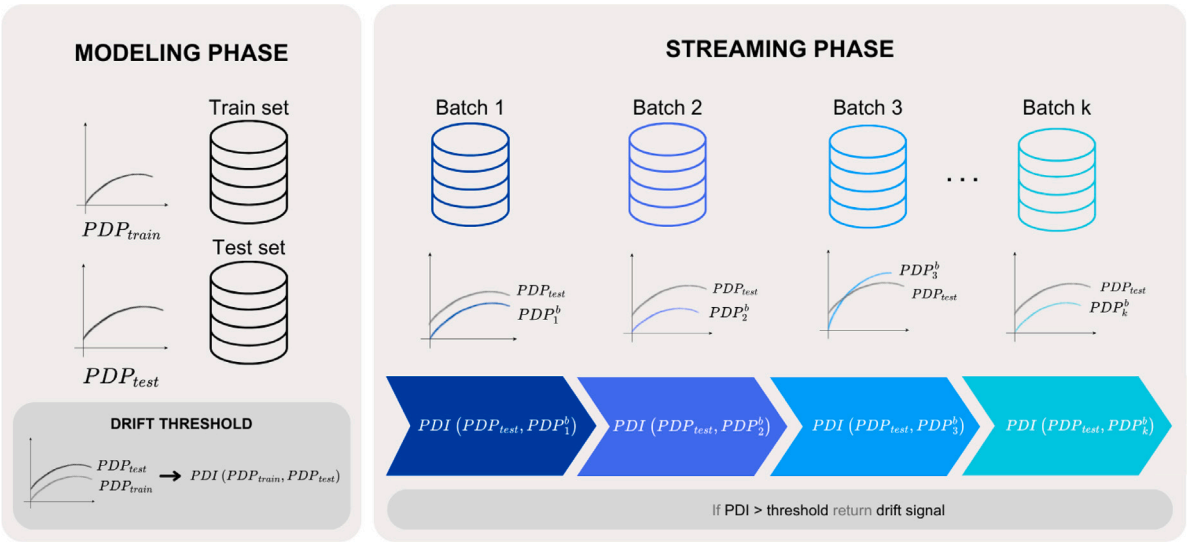


Fig. 2. The workflow of the Profile Drift Detection.

3.4. Page–Hinkley Test

Page–Hinkley Test (PH) is a method used to detect changes in mean values in data streams, similar to Cumulative Sums [26]. This method is designed to determine whether the average of the data obtained over a certain observation period remains constant. The key difference is the α forgetting factor, which weighs the observed values and the average, reducing the impact of older data. In the algorithm, initially $n = 1$, $x_{\text{ort}} = 0$, and $S = 0$ are set. For each incoming tth observation, the following operations are performed:

$$x_t = x_{\text{ort}} + \frac{x_t - x_{t-1}}{n} \tag{15}$$

$$S_t = \max\left(0, \alpha \cdot S_{t-1} + x_t - x - \delta\right) \tag{16}$$

If $S_t > \lambda$, it indicates the detection of a concept drift. PH can be used to test whether there is a change in the distribution of explanatory variables or the response variable, similar to the KSWIN test. For detecting concept drift, it is not necessary to wait for the actual value of the response variable predicted by the model. The PH test does not include an alert state.

3.5. Drift Detection Method

The Drift Detection Method (DDM) is used to detect changes in the response variable (class label) in binary classification problems [27]. This method is applied in situations where the data stream is continuous and is used to evaluate the model's performance each time a new example arrives. It is assumed that new observations arrive with true values in the form of (x_i, y_i). The model predicts the value $\hat{y}_i$ for the incoming observations as either Correct or Incorrect. The prediction values being Correct or Incorrect can be modeled with a Bernoulli distribution. When n observations are received, this distribution is modeled with a Binomial distribution. Let i be the number of examples:

Error Rate (p_i): The proportion of errors made by the model at the ith example. It is calculated over a specific set of examples and is the ratio of incorrectly classified examples to the total number of examples.

Standard Deviation (s_i): A value measuring the variance of the error rate at the ith example, calculated as $s_i = \sqrt{p_i(1-p_i)/i}$.

The error rate (p_i) is a random variable derived from Bernoulli trials and represented by a binomial distribution. For sufficiently large sample sizes, the binomial distribution approximates a normal distribution with the same mean and variance. The confidence interval for p_i at $1 - \alpha/2$ is calculated as $p_i \pm \alpha \cdot s_i$.

DDM monitors an increase in the error rate. If $(p_i + s_i)$ exceeds a certain threshold, it indicates concept drift. For a 95% confidence interval, a warning level is suggested as $p_i + s_i \geq p_{\min} + 2 \cdot s_{\min}$. For a 99% confidence interval, a drift level is suggested as $p_i + s_i \geq p_{\min} + 3 \cdot s_{\min}$. DDM is a model-independent, easily applicable algorithm. Since it only attempts to detect concept drift through the response variable, it cannot be used to detect virtual concept drift occurring in explanatory variables.

3.6. Early Drift Detection Method

The Early Drift Detection Method (EDDM) algorithm is similar to the DDM algorithm and is designed to detect concept drift in binary classification problems [28]. The main difference is that, while DDM calculates the error rate for each incoming (x_i, y_i) observation, EDDM calculates the distance between these errors.

The average distance between two errors p'_i and the standard deviation of this distance s'_i are computed and recorded. These values are saved when $p'_i + 2 \cdot s'_i$ reaches its maximum ($p'_{\max}$ and $s'_{\max}$). This maximum value indicates the moment when the model best represents the current concepts in the dataset. EDDM, like DDM, has two different threshold values for warning and drift levels:

Warning Level:

$$\frac{p'_i + 2 \cdot s'_i}{p'_{\max} + 2 \cdot s'_{\max}} < \alpha \tag{17}$$

where α is suggested to be 0.95.

Drift Level:

$$\frac{p'_i + 2 \cdot s'_i}{p'_{\max} + 2 \cdot s'_{\max}} < \beta \tag{18}$$

where β is suggested to be 0.90.

After detecting concept drift, the model is assumed to be retrained to fit the new concept, and $p'_{\max}$ and $s'_{\max}$ values are reset. EDDM is designed to detect slowly better-occurring concept drifts and performs well against sudden changes. Like DDM, EDDM detects concept drift based on the response variable, so it cannot be used to detect virtual concept drift occurring in the explanatory variables.

3.7. Profile drift detection

The combination of L2 distance, first-order derivative distance (L2Der), and the Partial Dependence Index (PDI) methods allows for a comprehensive assessment of various aspects of PDP comparisons [29]. L2 distance measures the overall difference and magnitude between two profiles. This method evaluates the structural similarity or differences between PDP shapes by considering squared differences between values across all points in the profiles. The L2Der compares the derivatives of profiles to analyze the rate and severity of changes within profiles. Comparing derivatives effectively identifies differences in slopes and local changes, aiding in understanding both the general shapes and dynamic behaviors of profiles. PDI focuses on whether profiles show an increase or decrease at the same data points, evaluating behavioral similarities by taking into account orientation changes between profiles. This index quantitatively expresses differences in profiles' trends and reveals opposing or similar behaviors. The use of these three methods together provides a holistic measure and analysis of the distance, change rates, and orientation differences between profiles. This allows for a deeper understanding of both the general similarity and behavior at different data points within PDPs. The difference in PDPs analyzed with these three metrics is referred to as PDD, illustrated in Fig. 2.

The three metrics used by PDD (PDI, L2, L2Der) can be set as parameters. In our prior studies [30,31], only the PDI metric was used to attempt to detect concept drift, but it was found to be insufficient on its own in some cases. PDD, on the other hand, incorporates both the L2 and L2Der metrics in addition to PDI. The thresholds set for the three metrics in Algorithm 1 should be appropriate for *acceptable* deviations.

In Algorithm 1, three metrics calculated over train–test data are compared with metrics obtained from the new stream. A large value for all three metrics indicates concept drift. The algorithm monitors the top-k most important variables simultaneously; drift is triggered when any variable exceeds all three thresholds. Upon detection, the model is retrained, the variable ranking is recalculated, and thresholds are updated using a pseudo-train/test split, allowing the method to adapt to evolving variable-importance structures. However, depending on the dataset's structure, the new data may be similarly shaped but offset along the vertical axis, which could lead to false negatives if the PDI metric is used. High metric values in the train and test sets can lead to false positives in new batches. Similarly, statistical tests have different sensitivity parameters. For example, the KSWIN test has parameters such as α, window size, and size. Similarly, PH, HDDM-W, HDDM-A, EDDM, and DDM each have three parameters. Changing the parameters of PDD and statistical tests affects the sensitivity of drift detection.

Algorithm 1 PDD_k: Top-k Variable Monitoring with Dynamic Thresholds

Input: Train and test sets, k (number of variables to monitor)
Output: $drift_count$, $accuracy$, $triggered$
$model \leftarrow$ Train model on the train set
$variables \leftarrow$ Top-k important variables via variable importance
for each v in $variables$ **do**
  Compute PDP for v on train ($profile1_train_v$) and test ($profile1_v$)
  $pdi_threshold_v, l2_threshold_v, l2der_threshold_v \leftarrow$ metrics($profile1_train_v$, $profile1_v$)
end for
for each new batch **do**
  $triggered \leftarrow \emptyset$
  for each v in $variables$ **do**
    $profile2_v \leftarrow$ Compute PDP for v on the new batch
    $PDI, L2, L2Der \leftarrow$ metrics($profile1_v$, $profile2_v$)
    if $PDI > pdi_threshold_v$ **and** $L2 > l2_threshold_v$ **and** $L2Der > l2der_threshold_v$ **then**
      $triggered \leftarrow triggered \cup \{v\}$
    end if
  end for
  if $triggered \neq \emptyset$ **then**
    $drift_count \leftarrow drift_count + 1$
    Retrain model on accumulated data
    $variables \leftarrow$ Recalculate top-k variables from updated model
    Split accumulated data into pseudo-train (80%) and pseudo-test (20%)
    for each v in $variables$ **do**
      Compute PDP for v on pseudo-train and pseudo-test
      Update $pdi_threshold_v, l2_threshold_v, l2der_threshold_v$ accordingly
    end for
    Recompute $profile1_v$ for each v on combined data
  end if
end for
return ($drift_count$, $accuracy$, $triggered$)

Note: PDD_C (Algorithm 1) is the special case where k=1, the monitored variable is fixed throughout (never recalculated), and thresholds remain unchanged after drift. In PDD_C, only the model and $profile1$ are updated upon drift detection.

Computational complexity. Let n denote the number of observations in the current batch, p the number of grid points used to evaluate each Partial Dependence Profile (PDP), T_{test} the cost of a single model prediction, d the number of explanatory variables, and b the number of permutation repetitions used in the variable-importance step. We write m for the number of monitored variables in the PDD variant, so that $m = 1$ for PDD_C and PDD_1, and $m = 2$ for PDD_2.

The baseline methods DDM, EDDM, HDDM-A, HDDM-W, and PH each maintain a constant number of scalar statistics, such as running error rates, cumulative sums, or exponentially weighted quantities. Therefore, they require $O(1)$ work per incoming observation, $O(n)$ work over a batch, and $O(1)$ auxiliary space. KSWIN is more expensive because the Kolmogorov–Smirnov test is applied to two sub-windows of size r at each update, leading to $O(r \log r)$ work per observation and $O(n)$ space for the sliding window.

For PDD, the dominant cost is the construction of PDPs. Evaluating one PDP for one monitored variable over p grid points requires querying the model on all n observations at each grid point, which costs $O(n \cdot p \cdot T_{\text{test}})$. The subsequent computation of the drift metrics (L2, L2Der, and, for classification, PDI) is only $O(p)$ per variable and is asymptotically negligible relative to PDP evaluation.

This immediately yields the no-drift cost of each variant. PDD_C monitors a single fixed variable and compares the current profile against fixed thresholds, so its per-batch complexity is $O(n \cdot p \cdot T_{\text{test}})$. PDD_1 also evaluates a single PDP per batch, hence it has the same no-drift cost, but differs operationally because the monitored variable may be updated after a detected drift. PDD_2 evaluates two PDPs at every drift-check cycle, so its no-drift complexity doubles to $O(2n \cdot p \cdot T_{\text{test}})$, which is equivalently $O(mn \cdot p \cdot T_{\text{test}})$ with $m = 2$.

When drift is detected, all PDD variants incur an additional retraining cost of $O(n \cdot d \cdot T_{\text{train}})$. For PDD_C, the post-drift update is relatively light: the model is retrained and the reference profile for the same variable is recomputed, so the total drift-case complexity remains

$$O(n \cdot p \cdot T_{\text{test}}) + O(n \cdot d \cdot T_{\text{train}}).$$

By contrast, PDD_1 and PDD_2 additionally recalculate variable importance after retraining, which costs $O(b \cdot n \cdot T_{\text{test}})$, and then rebuild pseudo-train/pseudo-test thresholds for the newly selected top-m variables. This threshold-refresh step requires recomputing PDPs for both pseudo-splits, adding another $O(mn \cdot p \cdot T_{\text{test}})$ term up to a constant factor. Hence, their drift-case cost is

$$O(mn \cdot p \cdot T_{\text{test}}) + O(n \cdot d \cdot T_{\text{train}}) + O(b \cdot n \cdot T_{\text{test}}),$$

with $m = 1$ for PDD_1 and $m = 2$ for PDD_2.

Table 1
Computational complexity of drift detection methods. n: batch size; r: KSWIN window size ($r \leq n$); p: PDP grid points; T_{test}: single prediction cost; d: number of features; b: permutation repetitions.

Method	Per-observation	Per batch	Space
DDM	$O(1)$	$O(n)$	$O(1)$
EDDM	$O(1)$	$O(n)$	$O(1)$
HDDM-A	$O(1)$	$O(n)$	$O(1)$
HDDM-W	$O(1)$	$O(n)$	$O(1)$
PH	$O(1)$	$O(n)$	$O(1)$
KSWIN	$O(r \log r)$	$O(nr \log r)$	$O(n)$
PDD_C (no drift)	–	$O(n \cdot p \cdot T_{\text{test}})$	$O(p)$
PDD_C (drift)	–	$O(n \cdot p \cdot T_{\text{test}} + n \cdot d \cdot T_{\text{train}})$	$O(p)$
PDD_1 (no drift)	–	$O(n \cdot p \cdot T_{\text{test}})$	$O(p)$
PDD_1 (drift)	–	$O(n \cdot p \cdot T_{\text{test}} + n \cdot d \cdot T_{\text{train}} + b \cdot n \cdot T_{\text{test}})$	$O(p)$
PDD_2 (no drift)	–	$O(2n \cdot p \cdot T_{\text{test}})$	$O(2p)$
PDD_2 (drift)	–	$O(2n \cdot p \cdot T_{\text{test}} + n \cdot d \cdot T_{\text{train}} + b \cdot n \cdot T_{\text{test}})$	$O(2p)$

Table 2
The characteristics of the benchmark dataset.

Dataset	Type	Task	#observations	#variables
`SEA`	Synthetic	Classification	50 000	4
`Hyperplane`	Synthetic	Classification	20 000	3
`NOAA`	Real	Classification	18 159	9
`Ozone`	Real	Classification	2 534	73
`Elec2`	Real	Classification	45 312	9
`Friedman`	Synthetic	Regression	20 000	11

Therefore, the computational gap between PDD_C and the adaptive variants PDD_1 and PDD_2 does not arise in ordinary no-drift batches; it appears mainly after a detected drift, when dynamic variable reselection and threshold recalibration are triggered. In practice, this means that PDD_1 preserves the same routine monitoring cost as PDD_C while offering greater adaptability, whereas PDD_2 trades an approximately twofold PDP-monitoring cost for broader drift coverage. A summary is provided in Table 1.

This overhead is the price of explainability: unlike scalar-statistic methods, PDD quantifies *how* the input–output relationship changes rather than merely *whether* it changes. For high-velocity streams where n is large or T_{test} is non-trivial, this cost can be reduced in practice through profile-grid compression, observation subsampling, or less frequent drift-check intervals.

4. Experiments

This section compares traditional methods and the PDD method using synthetic and real-world datasets. It also describes the characteristics of the benchmark dataset and the design of the experiment. The methods used in this part are ready to use in `datadriftR` package [32] in R.

4.1. Datasets

In the experiments, we used the generic synthetic and real-world datasets `SEA` [33], `Hyperplane` [34], `NOAA` [35], `Ozone` [36], `Elec2` [37], and `Friedman` [38]. The characteristics of these datasets are provided in Table 2.

4.2. Design of experiments

In the experiments, Logistic Regression (LR), Decision Tree (DT), and Random Forest (RF) models were used. However, for the `Friedman` dataset, linear regression was used instead of Logistic Regression.

For incoming data batches, it is determined whether the data is suitable for training the model. PDI, L2Der, and L2 are calculated from PDPs obtained from the train and test sets and are assigned as threshold values. If these three metrics exceed the specified thresholds, the new data batch is added to the train set, and the model is retrained. For the batches where the PDD method detects concept drift, PDP curves for the train–test sets and PDP graphs at the time of drift detection are included for each dataset. This process helps in detecting concept drift and allows the model to adapt to new data. Thus, the model is continuously updated and can adapt to changes in the data stream.

Statistical tests are conducted to verify whether the response variable in the incoming data batch is predicted correctly. For each new data batch, predicted values are obtained and tested against the actual values. If concept drift is detected in this data batch, similar incoming data is added to the train data, and the model is retrained.

For each experiment, Tables 3–9 show the concept drift detection method used, the number of batches, the size of the test set (equal to the batch size), the size of the training set, the number of detected drifts for each batch, and the model's performance on the training and testing sets. A larger batch size may lead to missed detections of drifts, while a smaller size can result in false detections. Therefore, determining the optimal batch size (or number of batches) is essential.

Additional information about the experiments, such as the number of observations in the batches (Table A.11), the most important variables in the batches (Table A.12), and the accuracy of the models in the batches (Table A.13), is given in Appendix.

All baseline drift detection methods (HDDM-A, HDDM-W, KSWIN, PH, DDM, EDDM) were used with their default parameter settings as implemented in the `datadriftR` R package [32]. No method-specific parameter tuning was performed, ensuring a fair and reproducible comparison. The experiments use 10, 20, and 30 batches per dataset to systematically evaluate the effect of batch granularity on drift detection, spanning a range from coarse to fine partitioning.

4.3. Results

In this section, the results of experiments conducted on the datasets `SEA`, `Hyperplane`, `NOAA`, `Ozone`, `Elec2`, and `Friedman` are summarized. The performance of the methods is evaluated in terms of accuracy and the number of detected drifts (`#drift`). The PDI metric was not used due to the similarity in shape, and the cutoff values for L2 and L2Der were set to five times the values calculated on the test set in the logistic regression models. All metrics were initialized on the test sets and used as cutoff values.

4.3.1. SEA

Table 3 presents the performance of various drift detection methods applied to three models (LR, DT, RF) over different batch sizes. Overall, accuracy values remain quite similar across methods, generally ranging between 0.82 and 0.85. However, there are notable differences in how many drift events each method detects.

For the LR model, methods such as KSWIN and EDDM register a very high number of drift detections (10, 20, and 30 or 10, 20, and 12 for 10, 20, and 30 batches, respectively). In contrast, PDD_C did not detect any drifts at any batch size, and other methods like PH and DDM reported minimal detections (e.g., PH: 1, 3, 3; DDM: 0, 3, 5). All PDD variants (PDD_1, PDD_2, PDD_3) similarly detect zero drifts. Unlike the `Hyperplane` case, where this led to accuracy degradation, here accuracy remains stable at 0.83–0.84 without retraining, confirming that the PDPs are genuinely unchanged and no actionable drift has occurred. This demonstrates that PDD's zero-detection outcome is appropriate when the model's functional relationship is preserved—the high drift counts from KSWIN and EDDM (up to 30) in this scenario likely represent false positives triggering unnecessary retraining.

While accuracy remains comparable across methods in the DT model, drift counts differ. HDDM-A and HDDM-W consistently detect

Table 3
The results of experiments on the SEA dataset.

Model	Method	#batch					
		10		20		30	
		Acc.	#drifts	Acc.	#drifts	Acc.	#drifts
LR	HDDM-A	0.8306	1	0.8462	2	0.8447	2
	HDDM-W	0.8279	3	0.8443	2	0.8407	2
	KSWIN	0.8292	10	0.8455	20	0.8422	30
	PH	0.8337	1	0.8447	3	0.8437	3
	DDM	0.8284	0	0.8460	3	0.8426	5
	EDDM	0.8292	10	0.8455	20	0.8381	12
	PDD_C	0.8284	0	0.8352	0	0.8392	0
	PDD_1	0.8284	0	0.8352	0	0.8392	0
	PDD_2	0.8284	0	0.8352	0	0.8392	0
	PDD_3	0.8284	0	0.8352	0	0.8392	0
DT	HDDM-A	0.8318	2	0.8400	2	0.8358	2
	HDDM-W	0.8248	3	0.8389	2	0.8354	2
	KSWIN	0.8294	10	0.8440	20	0.8376	29
	PH	0.8307	1	0.8424	3	0.8368	3
	DDM	0.8267	0	0.8432	1	0.8356	3
	EDDM	0.8291	9	0.8435	18	0.8340	8
	PDD_C	0.8230	1	0.8325	3	0.8349	8
	PDD_1	0.8267	0	0.8341	1	0.8357	2
	PDD_2	0.8267	0	0.8354	1	0.8374	3
	PDD_3	0.8238	7	0.8341	8	0.8374	3
RF	HDDM-A	0.8307	1	0.8417	2	0.8383	2
	HDDM-W	0.8259	3	0.8412	4	0.8371	2
	KSWIN	0.8267	10	0.8422	20	0.8396	28
	PH	0.8300	1	0.8418	3	0.8392	3
	DDM	0.8269	0	0.8427	3	0.8378	3
	EDDM	0.8267	10	0.8422	19	0.8391	13
	PDD_C	0.8283	5	0.8348	8	0.8377	18
	PDD_1	0.8267	1	0.8347	4	0.8383	3
	PDD_2	0.8267	1	0.8347	4	0.8363	1
	PDD_3	0.8249	3	0.8347	4	0.8363	1

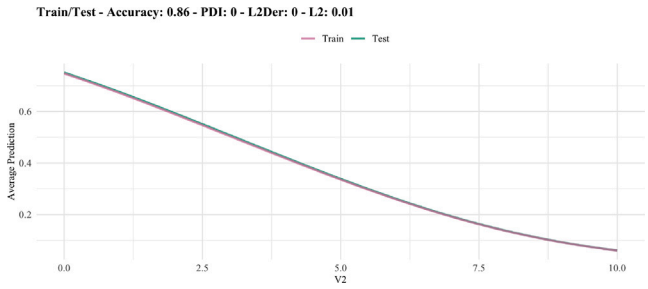


Fig. 3. PDPs of the Logistic Regression model trained on SEA dataset.

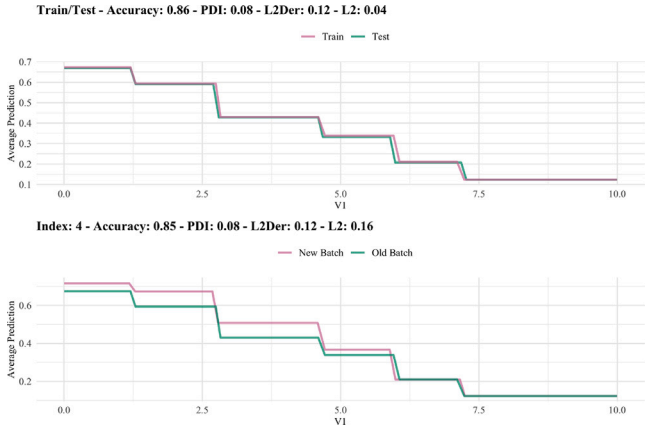


Fig. 4. PDPs of the Decision Tree model trained on SEA dataset.

2 or 3 drifts, whereas KSWIN and EDDM report much higher counts (around 10 to 29 drifts across different batch sizes). PDD_C, on the other hand, starts with a low detection count (1 drift at 10 batches) but increases to 8 drifts by the 30th batch, indicating a moderate sensitivity that grows with batch size. PDD_3, which monitors all three SEA features (V1, V2, V3), shows an inverse pattern: it detects 7 and 8 drifts at 10 and 20 batches but only 3 at 30 batches, with improved accuracy (0.8374 vs. 0.8349 for PDD_C). The dynamic threshold recalibration in PDD_3 progressively adapts to the stream, reducing sensitivity as the model stabilizes after initial drifts are addressed.

For the RF model, a similar pattern emerges. PDD_C detects 5 drifts at 10 batches, increasing to 8 and then 18 drifts as the batch size grows. This is intermediate compared to KSWIN and EDDM (which report 10–28 and 10–19–13 drifts, respectively) and the more conservative methods like DDM and PH (which remain within 0–3 drifts). Notably, PDD_1 and PDD_2 achieve the same or better accuracy (0.8383 and 0.8363 at 30 batches) with far fewer drift detections (3 and 1 drifts, respectively) compared to PDD_C's 18 drifts (0.8377). This indicates that PDD_C's fixed thresholds become overly sensitive as batches accumulate, triggering unnecessary retraining that does not improve accuracy. The dynamic threshold mechanism in PDD_1–PDD_3 effectively eliminates these false positives by recalibrating after each genuine drift, demonstrating that adaptive thresholds are as important as multi-variable monitoring for reducing computational overhead without sacrificing predictive performance.

In summary, while all methods maintain similar accuracy levels, there is a clear trade-off in drift detection sensitivity. Methods like KSWIN and EDDM are very aggressive in flagging drift events, potentially increasing the risk of false positives, especially in operational settings. Conversely, PDD_C exhibits a more conservative behavior in the LR model where it detects no drifts, but becomes overly sensitive in the RF model at larger batch sizes. The top-k variants with dynamic thresholds offer the best balance on this dataset: they correctly produce zero detections for LR (where no actionable drift exists), moderate detections for DT, and dramatically fewer false positives for RF (1–3 drifts vs. PDD_C's 18) while maintaining equivalent accuracy. This confirms that the dynamic threshold recalibration introduced in the top-k extension is particularly valuable for streams where the initial threshold may become miscalibrated over time.

In the LR model, no drift was detected—even during the 10th batch. As illustrated in Fig. 3, the PDPs for the train and test sets overlap almost perfectly, indicating that the functional relationship between the input variables and the response has remained stable. This complete overlap results in a PDI of 0, demonstrating that despite any shifts in data distribution, the model's predictive behavior has not been affected.

For the DT model with 10 batches, drift was detected in the 4th batch. As shown in Fig. 4, it is evident that the variable V1 plays a significant role in this drift. Specifically, within the range of 0 to 5, the contribution of V1 to the average response variable shows a noticeable decline. This suggests that within this particular range, the predictive influence of V1 weakens, which likely contributes to the observed drift. The reduction in V1's contribution implies that its relationship with the target variable has shifted in this batch, potentially due to underlying changes in the data distribution. Such a shift highlights the importance of monitoring variable contributions closely over time, as even subtle alterations can signal data drifts that may ultimately affect model performance.

The variable V2 has been identified as the most important variable in the RF model. Fig. 5 shows that V2 is the most important variable in the model's predictions. Specifically, within the range of 0 to 7.5, noticeable variations in the behavior of V2 are evident. These changes suggest that the influence of V2 on the model's output has shifted in these specific instances, potentially due to variations in the data

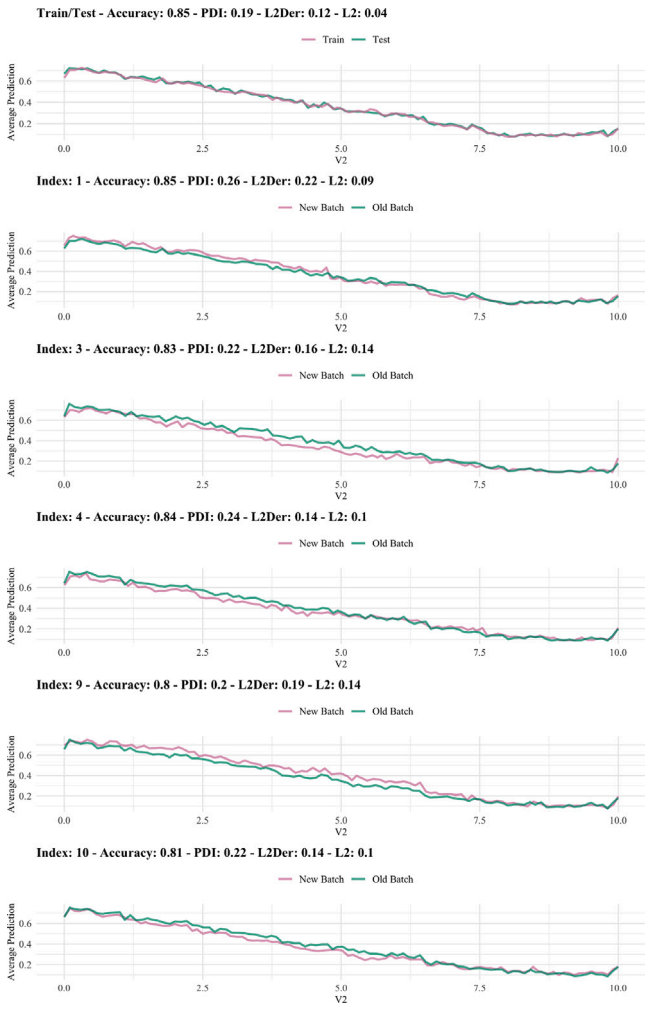


Fig. 5. PDPs of the Random Forest model trained on SEA dataset.

distribution during these batches. The observed changes in V2 highlight how shifts in important variables can significantly impact the model's performance and its ability to generalize well to incoming data. This further emphasizes the importance of monitoring key variables closely, as even small changes in their behavior can lead to drift detection and the need for model adaptation, particularly when dealing with real-time data streams.

4.3.2. *Hyperplane*

The results in Table 4 for the Hyperplane dataset reveal distinct performance trends across drift detection methods and models, with the PDD method showing mixed outcomes. The Hyperplane dataset is a synthetic stream where the decision boundary rotates incrementally over time, producing gradual concept drift. This characteristic makes it particularly challenging for threshold-based detectors: the per-batch change in PDP shape may be too small to exceed the drift threshold, yet the cumulative effect over many batches severely degrades accuracy.

An examination of the train and test set accuracies reveals a potential overfitting issue, Table A.13. For the LR model, most methods yield a high accuracy of approximately 0.86–0.87 at 10 batches. However, while methods such as HDDM-A, HDDM-W, KSWIN, PH, DDM, and EDDM maintain relatively high accuracy at 20 and 30 batches (ranging from about 0.80 to 0.83 at 20 batches and 0.75 to 0.83 at 30 batches),

Table 4
The results of experiments on the Hyperplane dataset.

Model	Method	#batch					
		10		20		30	
		Acc.	#drifts	Acc.	#drifts	Acc.	#drifts
LR	HDDM-A	0.8662	1	0.8204	2	0.7568	3
	HDDM-W	0.8377	1	0.8456	3	0.8152	4
	KSWIN	0.8639	9	0.8458	13	0.8285	18
	PH	0.8648	0	0.8187	1	0.7455	2
	DDM	0.8648	0	0.8071	1	0.7455	2
	EDDM	0.8648	0	0.8485	13	0.8348	5
	PDD_C	0.8648	0	0.6792	0	0.5219	0
	PDD_1	0.8648	0	0.6792	0	0.5219	0
	PDD_2	0.8648	0	0.6792	0	0.5219	0
DT	HDDM-A	0.8398	0	0.7670	3	0.6921	3
	HDDM-W	0.8398	0	0.7752	2	0.7316	4
	KSWIN	0.8520	9	0.8429	17	0.8162	19
	PH	0.8398	0	0.7014	1	0.6929	1
	DDM	0.8398	0	0.7014	1	0.7283	3
	EDDM	0.8496	10	0.8562	15	0.8379	11
	PDD_C	0.8435	1	0.8242	15	0.4924	1
	PDD_1	0.8474	1	0.7943	2	0.4552	0
	PDD_2	0.8474	1	0.7819	6	0.7629	13
RF	HDDM-A	0.7834	0	0.6809	1	0.6267	1
	HDDM-W	0.7794	2	0.7615	3	0.7536	6
	KSWIN	0.8219	10	0.8144	17	0.7948	22
	PH	0.7834	0	0.7033	1	0.6267	1
	DDM	0.7834	0	0.7290	3	0.6267	1
	EDDM	0.7834	0	0.8072	16	0.7957	24
	PDD_C	0.7831	1	0.7189	4	0.5099	0
	PDD_1	0.7831	0	0.6971	1	0.5099	0
	PDD_2	0.8193	4	0.7433	6	0.6617	9

PDD_C experiences a dramatic drop—from 0.8648 at 10 batches to 0.6792 at 20 batches and further to 0.5219 at 30 batches. Notably, PDD does not detect any drift events in the LR model across all batch sizes, which may indicate that it is overly conservative or insensitive to drifts when the model's predictive behavior is stable, even if the overall performance degrades with larger batch sizes. Crucially, this failure persists across all PDD variants (PDD_1, PDD_2), which also detect zero drifts. Note that PDD_3 is omitted for this dataset because only two features (feature_1, feature_2) are informative, making the third monitored variable redundant. Because Logistic Regression produces smooth, monotone PDPs for the Hyperplane data, the incremental rotation of the decision boundary causes only negligible per-batch changes in PDP shape—each change remains below the threshold even though the accumulated drift is substantial. This represents a fundamental limitation of PDD under purely gradual drift with linear models: the method requires that drift manifests as a detectable PDP shape change within a single batch interval. Performance-based methods such as KSWIN and EDDM avoid this issue because they directly monitor prediction errors, which degrade steadily even when PDP shape changes are imperceptible.

In the case of the DT model, accuracy levels at 10 batches are fairly consistent across methods (around 0.84), yet divergence emerges as the number of batches increases. While methods like KSWIN and EDDM continue to yield high accuracies (up to 0.8429 and 0.8562, respectively, at 20 batches, and 0.8162 and 0.8379 at 30 batches), PDD_C shows an unusual pattern — detecting only 1 drift at 10 batches but spiking to 15 detected drifts at 20 batches — with accuracy falling to 0.8242 at 20 batches and dropping to 0.4924 at 30 batches. The top-k extension substantially improves this situation: PDD_2 detects 13 drifts at 30 batches and maintains accuracy at 0.7629—a dramatic improvement over PDD_C's 0.4924. This occurs because the DT model's non-linear splits amplify PDP shape changes when both feature_1 and feature_2 are monitored simultaneously, enabling drift signals that remain sub-threshold for a single variable to be captured across the pair.

For the RF model, all methods report lower accuracies compared to the LR and DT models. While KSWIN and EDDM achieve relatively high accuracies (0.8144 and 0.8072 at 20 batches, and 0.7948 and 0.7957 at 30 batches, respectively), PDD_C registers moderate accuracy at 10 batches (0.7831) but then declines to 0.7189 at 20 batches and further plummets to 0.5099 at 30 batches with zero drifts detected. PDD_2 offers a clear improvement: it detects 4, 6, and 9 drifts across 10, 20, and 30 batches, achieving accuracies of 0.8193, 0.7433, and 0.6617, respectively. At 30 batches, this represents a 15 percentage-point gain over PDD_C (0.6617 vs. 0.5099). The RF model's severe overfitting (training accuracy ≈1.0 vs. test accuracy ≈0.48 at 30 batches, Table A.13) amplifies PDP differences between training and test sets, which paradoxically helps PDD_2 detect drift through the amplified non-linear PDP distortions—an effect absent in the linear LR model.

Overall, methods like KSWIN and EDDM consistently register a higher number of drift detections, underscoring their sensitivity to changes in the data stream; however, this heightened sensitivity could potentially lead to more false positives. In contrast, HDDM-A and HDDM-W tend to detect fewer drifts, and methods such as PH and DDM offer moderate performance in both accuracy and drift detection. The PDD method, in particular, demonstrates a mixed profile: it preserves high accuracy in the LR model at 10 batches but suffers a pronounced accuracy decline and fails to detect any drifts as the batch size increases. For DT and RF models, PDD_C's drift detection is inconsistent and generally lower compared to its counterparts, with a notable drop in accuracy at higher batch sizes. The top-k variant partially addresses this: PDD_2 recovers substantial accuracy for DT (+27 pp at 30 batches) and RF (+15 pp at 30 batches) by detecting more drift events through two-variable monitoring. However, the LR case remains an acknowledged limitation where gradual drift does not produce per-batch PDP changes exceeding the threshold for any number of monitored variables.

These findings suggest that the effectiveness of the PDD method in the `Hyperplane` dataset is highly dependent on both the model type and the batch size. While its conservative nature may reduce the incidence of false positive drift detections, it also raises concerns about its ability to capture actual drifts when the model's performance is already compromised. Practitioners facing gradually drifting streams should consider combining PDD with a lightweight performance monitor, e.g., an accuracy-based trigger, to complement the profile-based approach, or employing shorter batch intervals that increase the signal-to-noise ratio of per-batch PDP changes.

4.3.3. NOAA

The results for the `NOAA` dataset in Table 5 show that while overall accuracy levels across methods remain relatively stable—with LR and RF models achieving accuracies in the mid- to high 0.70 and DT models around 0.73–0.74, the number of drift detections varies considerably between methods and models.

For the LR model, PDD_C maintains a consistent accuracy of approximately 0.770 across all batch sizes. However, it flags very few drift events, with only 1 drift at 10 batches, 2 at 20 batches, and 4 at 30 batches. In contrast, more sensitive methods such as KSWIN and EDDM detect between 10 and 27 drifts (KSWIN) or 10 and 30 drifts (EDDM) across the same batch sizes. PDD_2 and PDD_3 increase sensitivity progressively, detecting 3, 2, and 10 drifts at 10, 20, and 30 batches with improved accuracy (0.7744 vs. 0.7705 for PDD_C at 30 batches). This is consistent with the variable importance instability observed in the LR model, where the top variable shifts from `attr8` (10 batches) to `attr4` (20 batches) and then to `attr2` (30 batches)—monitoring multiple variables allows PDD to capture drifts that a fixed single-variable approach would miss.

The DT model exhibits a similar trend in terms of accuracy, where PDD_C detects only 2 drifts at 10 batches, none at 20 batches, and 3 at 30 batches—again, far fewer than the consistently higher counts produced by KSWIN and EDDM (which report 10, 20, and 30 drifts, respectively). PDD_3 provides markedly better results for the DT model, detecting 7, 13, and 13 drifts with the highest accuracy among PDD variants (0.7407, 0.7382, and 0.7414), compared to PDD_C (0.7264, 0.7403, 0.7352). This improvement is particularly notable at 30 batches, where PDD_3 achieves accuracy comparable to KSWIN (0.7406) while detecting fewer drifts (13 vs. 30), demonstrating a better balance between sensitivity and false-positive control.

The RF model presents a different pattern. Although its overall accuracy stays in the 0.78–0.79 range, PDD_C's drift detection is notably inconsistent: it detects 5 drifts at 10 batches, 0 drifts at 20 batches, and then jumps to 22 drifts at 30 batches. This inconsistency is particularly striking when compared to KSWIN (which detects 10, 18, and 26 drifts) and EDDM (with 10, 20, and 30 drifts). Given that the RF model is known to overfit — evidenced by an almost perfect training accuracy contrasted with its moderate test accuracy — this erratic behavior in PDD_C's drift detection may be influenced by the overfitting, which can distort the Partial Dependence Profile (PDP) comparisons used by PDD. The multi-variable variants partially mitigate this issue: PDD_2 and PDD_3 detect 4, 1, and 10–12 drifts, providing a more gradual and controlled detection pattern compared to PDD_C's abrupt 0-to-22 jump. However, the RF overfitting remains a limiting factor—when training accuracy approaches 1.0, PDPs from the training set become overly confident and diverge sharply from test-set PDPs, inflating the distance metrics. This highlights that PDD's reliability depends on the underlying model being well-calibrated, and practitioners should consider regularization or ensemble pruning when applying PDD to models prone to overfitting.

Other methods show their characteristics: PH remains steady with 3 drifts for LR and RF (and 2 for DT) across all batch sizes, while HDDM-A and HDDM-W detect a moderate number of drifts that vary with both model and batch size.

Table 5
The results of experiments on the `NOAA` dataset.

Model	Method	#batch					
		10		20		30	
		Acc.	#drifts	Acc.	#drifts	Acc.	#drifts
LR	HDDM-A	0.7736	5	0.7696	7	0.7777	3
	HDDM-W	0.7736	7	0.7712	12	0.7799	12
	KSWIN	0.7755	10	0.7755	20	0.7788	27
	PH	0.7707	3	0.7680	3	0.7792	3
	DDM	0.7716	4	0.7746	10	0.7715	10
	EDDM	0.7755	10	0.7755	20	0.7780	30
	PDD_C	0.7700	1	0.7694	2	0.7705	4
	PDD_1	0.7713	2	0.7680	1	0.7784	4
	PDD_2	0.7729	3	0.7698	2	0.7744	10
	PDD_3	0.7729	3	0.7698	2	0.7744	10
DT	HDDM-A	0.7293	8	0.7435	9	0.7440	8
	HDDM-W	0.7385	10	0.7345	16	0.7385	23
	KSWIN	0.7385	10	0.7334	20	0.7406	30
	PH	0.7402	2	0.7387	2	0.7276	2
	DDM	0.7422	3	0.7334	20	0.7398	21
	EDDM	0.7385	10	0.7334	20	0.7406	30
	PDD_C	0.7264	2	0.7403	0	0.7352	3
	PDD_1	0.7264	2	0.7398	2	0.7306	5
	PDD_2	0.7318	2	0.7369	10	0.7307	8
	PDD_3	0.7407	7	0.7382	13	0.7414	13
RF	HDDM-A	0.7796	0	0.7812	5	0.7770	2
	HDDM-W	0.7877	4	0.7805	11	0.7824	9
	KSWIN	0.7914	10	0.7894	18	0.7922	26
	PH	0.7877	3	0.7784	3	0.7792	3
	DDM	0.7809	1	0.7916	20	0.7829	8
	EDDM	0.7914	10	0.7916	20	0.7941	30
	PDD_C	0.7881	5	0.7676	0	0.7913	22
	PDD_1	0.7875	3	0.7676	0	0.7863	6
	PDD_2	0.7880	4	0.7676	1	0.7856	10
	PDD_3	0.7880	4	0.7676	1	0.7871	12

In summary, while PDD_C yields stable accuracy and a controlled number of drift detections for the LR and DT models, its performance in the RF model is inconsistent—likely due to overfitting effects. The top-k variants (PDD_2, PDD_3) address this limitation in two ways: (i) they smooth out erratic detection patterns by distributing the monitoring across multiple variables, and (ii) they achieve higher accuracy in the DT model by enabling more timely retraining triggered by drifts in secondary variables. This suggests that PDD might be less reliable in environments where models are prone to overfitting, as it can under-detect or inconsistently detect drift events compared to more sensitive methods like KSWIN and EDDM, which, although potentially generating more false positives, offer a more consistent detection framework across different batch sizes and models.

4.3.4. Ozone

The experimental results for the Ozone dataset, as presented in Table 6, offer a detailed picture of both drift detection performance and variable importance dynamics across different models and batch sizes. Overall, accuracy remains high for all models, yet the number of drift detections varies noticeably depending on the method employed. For example, in the LR model, accuracy starts at approximately 0.93 for 10 batches, declines slightly to around 0.89 for 20 batches, and then reaches about 0.87 for 30 batches. At the same time, while methods like HDDM-A, HDDM-W, and PH report minimal drift events — often zero or just a couple — the more sensitive techniques such as KSWIN, DDM, and EDDM tend to detect more drifts, with counts ranging from 3 to 6. The PDD_C method, in particular, shows moderate sensitivity in the LR model by detecting 2 drifts for both 10 and 20 batches and 3 drifts for 30 batches. The multi-variable variants PDD_2 and PDD_3 are notably more sensitive, detecting up to 10 drifts at 20 batches and 12–17 drifts at 30 batches, with improved accuracy (0.901 and 0.913 vs. 0.872 for PDD_C at 30 batches). This demonstrates the benefit of monitoring multiple variables in high-dimensional datasets where the most important variable frequently changes.

In the DT model, accuracies are consistently high, hovering around 0.94 at 10 batches, with slight variations as batch size increases. While some methods register only a few drift events, the PDD_C method exhibits an interesting pattern: it does not detect any drift at 10 batches, detects a single drift at 20 batches, and then experiences a sudden increase to 10 drift detections at 30 batches. This abrupt change may be reflective of an evolving decision boundary as the model processes more data. PDD_2 and PDD_3, which monitor 2 and 3 variables simultaneously, detect 11 and 15 drifts at 20 batches with improved accuracy (0.923) compared to PDD_C (0.892), demonstrating that multi-variable monitoring captures drifts earlier and enables timely retraining. The RF model, on the other hand, maintains high accuracy — often exceeding 0.94 — but shows inconsistent drift detection with PDD_C, which detects 5 drifts at 10 batches, none at 20 batches, and then jumps to 12 drifts at 30 batches. Such variability in the RF model might be linked to its inherent complexity or even issues like overfitting, which can affect how subtle changes in the model's predictive profile are interpreted by drift detection methods.

Adding another layer to this analysis, the variable importance table (Table A.12) provides insights into how the most critical variables evolve with batch size across different datasets and models. In the Ozone dataset, for instance, the RF model shifts its focus from variable V56 at 10 batches to V61 at 20 and 30 batches, while the LR model moves from V25 to V19, and the DT model transitions from V31 to V36 as the batch size increases. These changes suggest that as more data is incorporated, the models may be re-weighting the influence of certain variables, which could lead to alterations in the shape of the PDPs and, consequently, affect drift detection outcomes. This instability in variable ranking is precisely the scenario where the top-k extension (PDD_1, PDD_2, PDD_3) provides the most benefit: by monitoring multiple variables, the method avoids missing drifts caused by variables that were not initially ranked first but later become influential.

Table 6
The results of experiments on the Ozone dataset.

Model	Method	#batch					
		10		20		30	
		Acc.	#drifts	Acc.	#drifts	Acc.	#drifts
LR	HDDM-A	0.9294	0	0.8942	2	0.8116	0
	HDDM-W	0.9294	0	0.8907	1	0.8526	1
	KSWIN	0.9396	3	0.9164	2	0.9144	5
	PH	0.9294	0	0.8502	0	0.8116	0
	DDM	0.9406	4	0.9155	6	0.8706	2
	EDDM	0.9396	2	0.9183	4	0.8842	6
	PDD_C	0.9300	2	0.9126	2	0.8722	3
	PDD_1	0.9288	1	0.9121	6	0.9027	7
	PDD_2	0.9288	1	0.9180	10	0.9009	17
	PDD_3	0.9288	1	0.9180	10	0.9131	12
DT	HDDM-A	0.9487	0	0.9101	1	0.9036	2
	HDDM-W	0.9487	0	0.9101	1	0.9122	2
	KSWIN	0.9444	3	0.9242	4	0.9149	5
	PH	0.9487	0	0.8911	0	0.8725	0
	DDM	0.9358	7	0.9149	6	0.9329	4
	EDDM	0.9412	2	0.9164	4	0.9185	5
	PDD_C	0.9435	0	0.8915	1	0.9140	10
	PDD_1	0.9435	0	0.8911	0	0.8964	1
	PDD_2	0.9435	0	0.9232	11	0.8964	1
	PDD_3	0.9394	1	0.9233	15	0.8964	1
RF	HDDM-A	0.9508	0	0.9427	1	0.9374	0
	HDDM-W	0.9508	0	0.9427	1	0.9401	1
	KSWIN	0.9529	1	0.9383	4	0.9437	3
	PH	0.9508	0	0.9422	0	0.9374	0
	DDM	0.9513	2	0.9427	2	0.9437	3
	EDDM	0.9524	2	0.9441	3	0.9437	3
	PDD_C	0.9459	5	0.9422	0	0.9455	12
	PDD_1	0.9453	1	0.9422	0	0.9423	1
	PDD_2	0.9453	1	0.9413	11	0.9441	1
	PDD_3	0.9453	1	0.9427	9	0.9441	1

When considering the interplay between drift detection and variable importance, it becomes clear that methods that rely on comparing the behavior of key variables, such as the PDD approach, might register drift events not solely due to shifts in data distribution but also due to changes in the relative importance of those variables. The sudden increase in drift detections by PDD_C in the DT model at 30 batches could be partly attributed to its change in the most influential variable from V31 to V36. Similarly, the inconsistent detection pattern observed in the RF model with PDD_C might be related to how the model's primary variables are adjusted as more batches are processed.

In summary, the results indicate that while high accuracy is maintained across models in the Ozone dataset, the sensitivity of drift detection methods varies significantly. Conservative methods tend to produce fewer detections, whereas more sensitive ones capture a higher number of drift events, which may lead to false positives. The PDD_C method, although demonstrating moderate performance in the LR model, exhibits inconsistent behavior in the DT and RF models — particularly as batch size increases — potentially due to changes in variable importance or model overfitting. The multi-variable variants (PDD_2, PDD_3) mitigate this inconsistency by tracking multiple features, enabling more stable drift detection even when the dominant variable shifts across batches. This analysis underscores the complexity of drift detection in dynamic environments and highlights the importance of monitoring both model accuracy and feature relevance over time to refine drift detection strategies effectively.

4.3.5. Elec2

The results for the Elec2 dataset reveal distinct performance patterns across drift detection methods, models, and batch sizes. In particular, the PDD method exhibits a balanced approach in terms of both accuracy and the number of drift detections. For the Logistic Regression model, PDD_C achieves an accuracy of 0.6740 at 10 batches, which increases to 0.7349 at 20 batches and further to 0.7402 at 30

Table 7
The results of experiments on the Elec2 dataset.

Model	Method	#batch					
		10		20		30	
		Acc.	#drifts	Acc.	#drifts	Acc.	#drifts
LR	HDDM-A	0.6983	10	0.6836	20	0.7099	30
	HDDM-W	0.6983	10	0.6836	20	0.7099	30
	KSWIN	0.6983	10	0.6836	20	0.7099	30
	PH	0.6983	10	0.7342	13	0.7390	16
	DDM	0.6983	10	0.6836	20	0.7099	30
	EDDM	0.6983	10	0.6836	20	0.7099	30
	PDD_C	0.6740	8	0.7349	4	0.7402	12
	PDD_1	0.7243	2	0.7321	4	0.7308	11
	PDD_2	0.7243	2	0.7262	5	0.7359	10
	PDD_3	0.7243	2	0.7262	5	0.6862	11
DT	HDDM-A	0.7255	10	0.7409	20	0.7450	30
	HDDM-W	0.7255	10	0.7409	20	0.7450	30
	KSWIN	0.7255	10	0.7409	20	0.7450	30
	PH	0.7215	9	0.7417	15	0.7502	16
	DDM	0.7255	10	0.7409	20	0.7450	30
	EDDM	0.7255	10	0.7409	20	0.7450	30
	PDD_C	0.7253	1	0.7392	1	0.7429	5
	PDD_1	0.7253	1	0.7392	1	0.7441	4
	PDD_2	0.7234	1	0.7061	3	0.7442	10
	PDD_3	0.7308	3	0.7061	3	0.7472	16
RF	HDDM-A	0.7029	10	0.7243	20	0.7374	30
	HDDM-W	0.7029	10	0.7243	20	0.7374	30
	KSWIN	0.7029	10	0.7243	20	0.7374	30
	PH	0.6992	8	0.7103	11	0.7368	16
	DDM	0.7029	10	0.7250	16	0.7253	17
	EDDM	0.7029	10	0.7243	20	0.7374	30
	PDD_C	0.7093	3	0.7311	0	0.7449	3
	PDD_1	0.7204	1	0.7311	0	0.7449	2
	PDD_2	0.6897	5	0.7014	3	0.7355	15
	PDD_3	0.6897	5	0.7014	3	0.7274	16

batches. Similarly, for the Decision Tree model, the accuracy values are 0.7253, 0.7392, and 0.7429 at 10, 20, and 30 batches, respectively. The Random Forest model shows accuracies of 0.7093, 0.7311, and 0.7449 across the three batch sizes.

When comparing drift detection counts, PDD maintains a moderate level of sensitivity. In the LR model, PDD_C detects 8 drifts at 10 batches, 4 at 20 batches, and 12 at 30 batches. For the DT model, the method is even more conservative, with just 1 drift detected at both 10 and 20 batches and an increase to 5 drifts at 30 batches. In the RF model, PDD_C identifies 3 drifts at 10 batches, no drift at 20 batches, and 3 drifts at 30 batches. This controlled drift detection contrasts with methods such as HDDM-A, HDDM-W, KSWIN, DDM, and EDDM, which consistently flag a higher number of drifts. In the LR model, these methods uniformly detect 10 drifts across all batch sizes, while the PH method shows an increasing trend with up to 16 drifts in the DT and RF models at 30 batches. Notably, for the DT and RF models, PDD_2 and PDD_3 progressively detect more drifts as batch count increases (up to 16 drifts at 30 batches for RF), approaching the sensitivity of baseline methods while maintaining comparable or better accuracy.

Accuracy remains fairly robust across all methods; the LR model maintains performance above 0.69, and the DT and RF models consistently achieve accuracies above 0.70. Such consistency in predictive performance is further underscored by the observation that the base test and train accuracies are high across all models. An additional key insight comes from the variable importance analysis: for the Elec2 dataset, the attribute `nswprice` is identified as the most important variable for LR, DT, and RF models across all batch sizes. This stability in variable importance indicates that `nswprice` is a critical factor driving model predictions, which in turn may contribute to more reliable drift detection outcomes when this feature remains consistently influential.

Table 8
Runtime comparison on the Elec2 dataset (RF model, 10 batches). N: subsample size for PDP computation; G: quantile grid resolution ("–" denotes the default full grid). Speedup is relative to PDD_3 with default settings.

Method	Configuration	Acc.	#drifts	Time (s)	Speedup
PDD — default ($N = 100$, G = full)					
PDD_1	$N = 100$	0.7204	1	1.90	6.5×
PDD_2	$N = 100$	0.6908	4	6.05	2.0×
PDD_3	$N = 100$	0.6976	7	12.26	1.0×
PDD — grid compression ($G = 10$)					
PDD_1	$N = 100$, $G = 10$	0.7236	1	1.18	10.4×
PDD_2	$N = 100$, $G = 10$	0.6947	3	3.28	3.7×
PDD_3	$N = 100$, $G = 10$	0.6947	3	3.44	3.6×
PDD — combined compression ($N = 30$, $G = 10$)					
PDD_1	$N = 30$, $G = 10$	0.7236	1	1.07	11.5×
PDD_2	$N = 30$, $G = 10$	0.7236	1	1.27	9.7×
PDD_3	$N = 30$, $G = 10$	0.6961	2	2.52	4.9×
Baseline methods					
HDDM_A	–	0.7029	10	8.00	1.5×
HDDM_W	–	0.7029	10	7.52	1.6×
KSWIN	–	0.7029	10	7.09	1.7×
PH	–	0.6992	8	5.18	2.4×
DDM	–	0.7029	10	7.53	1.6×
EDDM	–	0.7029	10	7.07	1.7×

Computational efficiency. The theoretical complexities in Table 1 are empirically validated below. Table 8 reports the wall-clock runtime of PDD and baseline methods on the Elec2 dataset (Random Forest, 10 batches). Two compression strategies are evaluated to reduce PDD's cost: (i) reducing the subsample size N used for PDP computation (default $N = 100$), and (ii) replacing the full evaluation grid with a G-point quantile grid.

Grid compression alone ($G = 10$) reduces PDD_3 runtime from 12.3 s to 3.4 s — a 3.6× speedup — with negligible accuracy loss (from 0.698 to 0.695). Combining both levers ($N = 30$, $G = 10$) yields a 4.9× speedup (2.5 s) while accuracy remains within 0.2% of the default configuration. Notably, baseline methods require 7–8 s on this dataset, which is slower than PDD_1 (1.9 s) and comparable to compressed PDD_3 (2.5 s). This is because baselines trigger 10 retraining events each, and model fitting dominates their total runtime. PDD produces fewer but more targeted drift signals while providing interpretable diagnostics about which variables caused the drift. These results confirm that the *compress then explain* approach [39] effectively mitigates PDD's computational overhead, making it competitive with traditional detectors while preserving explainability.

Overall, the PDD method for the Elec2 dataset demonstrates a balanced performance by maintaining competitive accuracy while detecting a moderate and controlled number of drifts. Its more conservative nature in comparison to other methods can be advantageous in operational scenarios where minimizing false positives is essential. Furthermore, the consistent identification of `nswprice` as the key variable across models reinforces the importance of focused variable analysis in understanding model behavior and drift dynamics.

For the RF model applied to the Elec2 dataset, drift was detected in the 10th batch. In Fig. 6, the `nswprice` variable, which is the most important variable, shows noticeable changes between 0 and 0.1. In this range, the contribution of `nswprice` to the model's predictions shifts, leading to drift detection by the PDD. These variations in `nswprice` highlight how its influence on the model differs in the 10th batch, potentially impacting the model's performance.

4.3.6. Friedman

The results for the Friedman dataset in Table 9, which pertains to a regression task, exhibit distinct performance patterns across various drift detection methods, models, and batch sizes. For the regression task, only the L2 and L2Der metrics are used as drift gates (PDI is disabled because regression PDPs tend to have similar shapes, making

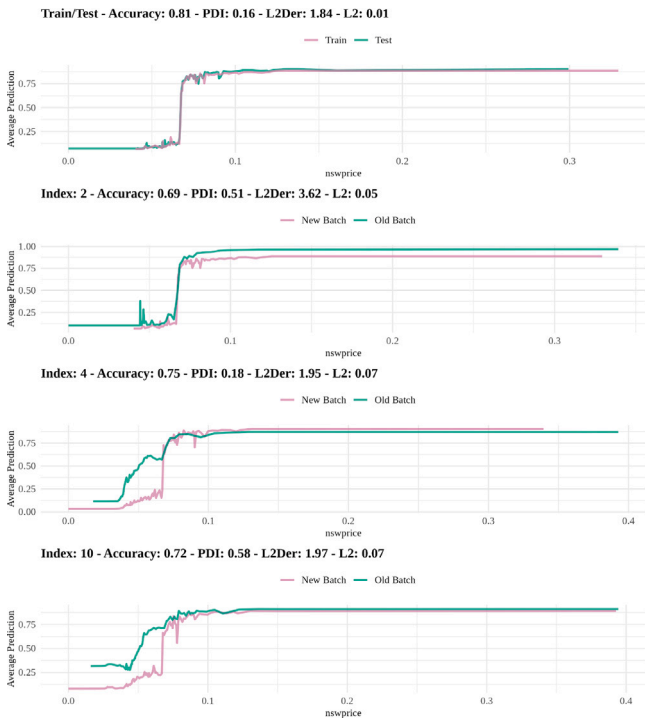


Fig. 6. PDPs of the Random Forest model trained on `Elec2` dataset.

PDI uninformative). The relaxation coefficient is set to $\lambda = 5$ to account for the higher natural variance in regression profiles. Focusing on the PDD method, it demonstrated a balanced approach in terms of both RMSE values and the number of detected drifts. Specifically, for the LR model, PDD_C achieved RMSE values of 2.8332, 2.8073, and 2.7924 across 10, 20, and 30 batches, respectively, while detecting 3, 1, and 1 drifts. In the DT model, PDD_C maintained stable RMSE values of 3.3460, 3.2936, and 3.2560, with drift detections of 3, 2, and 2 drifts across the same batch sizes. Similarly, for the RF model, the RMSE values reported by PDD_C are 2.0112, 1.9320, and 2.0467, with drift detections of 8, 17, and 4 drifts, respectively. This consistent performance indicates that PDD effectively maintains model accuracy while controlling the number of drift detections, thereby reducing the likelihood of unnecessary drift alarms. Monitoring multiple variables (PDD_2 and PDD_3) generally increases drift detection counts — particularly for RF where PDD_3 detects 7, 17, and 24 drifts — yielding lower RMSE values due to more frequent retraining.

In comparison, the KSWIN method consistently detected a higher number of drifts across all models and batch sizes. For instance, in the LR model, KSWIN identified 10, 19, and 30 drifts with RMSE decreasing slightly from 2.8356 to 2.7835 as the number of batches increased. Similarly, in the DT and RF models, KSWIN detected 10, 19, and 23 drifts and 10, 20, and 25 drifts, respectively, with RMSE remaining relatively stable or showing minor improvements. This high sensitivity allows KSWIN to capture more drift events, which can be advantageous for detecting subtle changes in the data stream but may also lead to a higher risk of false positives.

The PH method exhibited moderate sensitivity, detecting fewer drifts compared to KSWIN but more than PDD. In the LR model, PH detected 5, 10, and 9 drifts with RMSE improving from 2.8328 to 2.7755. The DT model showed consistent drift detections of 9, 16, and 16 drifts, while the RF model detected 3, 5, and 7 drifts across the batch sizes, all while maintaining stable RMSE values. PH strikes a balance between sensitivity and stability, making it suitable for scenarios where controlled drift detection is preferred without the excessive alerts that KSWIN might produce.

Table 9
The results of experiments on the `Friedman` dataset.

Model	Method	#batch					
		10		20		30	
		RMSE	#drifts	RMSE	#drifts	RMSE	#drifts
LR	KSWIN	2.8356	10	2.8007	19	2.7835	30
	PH	2.8328	5	2.7974	10	2.7755	9
	PDD_C	2.8332	3	2.8073	1	2.7924	1
	PDD_1	2.8438	4	2.8073	1	2.7943	1
	PDD_2	2.8430	5	2.8079	2	2.7874	27
	PDD_3	2.8422	6	2.8001	18	2.7830	28
DT	KSWIN	3.3364	10	3.2765	19	3.2875	23
	PH	3.3596	9	3.2813	16	3.2710	16
	PDD_C	3.3460	3	3.2936	2	3.2560	2
	PDD_1	3.3367	1	3.2287	4	3.2846	8
	PDD_2	3.3494	4	3.2255	9	3.2475	20
	PDD_3	3.3264	8	3.2799	15	3.2765	24
RF	KSWIN	1.9987	10	1.9282	20	1.9244	25
	PH	2.0574	3	2.0654	5	2.0139	7
	PDD_C	2.0112	8	1.9320	17	2.0467	4
	PDD_1	2.0503	1	1.9772	10	1.9993	18
	PDD_2	2.0084	9	1.9535	16	1.9712	17
	PDD_3	2.0054	7	1.9415	17	1.9433	24

Across all methods and models, the most important variable consistently identified was `feature_4`. This uniformity suggests that `feature_4` plays a pivotal role in influencing model performance, regardless of batch size or the drift detection method employed. This stability in variable ranking explains why PDD_C and PDD_1 produce nearly identical results on this dataset: since the top variable does not change after retraining, dynamic variable selection offers no advantage over the fixed-variable strategy.

In summary, the PDD method demonstrated balanced effectiveness for the `Friedman` regression task by maintaining reasonable RMSE values and controlling the number of detected drifts across different batch sizes. While PDD offers a more conservative approach to drift detection, reducing the likelihood of false positives, it may be less sensitive compared to methods like KSWIN, which, although more sensitive, carry a higher risk of false alarms. The PH method provides an intermediate level of sensitivity, offering a stable drift detection mechanism for controlled environments. Additionally, the consistent identification of `feature_4` as the most important variable across all models and batch sizes emphasizes its critical role in model performance, suggesting that adaptive variable selection strategies could further enhance drift detection and overall model reliability in dynamic data environments.

Threshold sensitivity. PDD's drift condition requires that the active metrics simultaneously exceed their respective thresholds, where each threshold equals the baseline distance (from the initial train–test PDP comparison) multiplied by a relaxation coefficient λ. For classification tasks, all three metrics (PDI, L2, L2Der) are used with $\lambda = 1$; the conjunctive PDI gate already suppresses false positives from shape-preserving vertical shifts. For regression, only L2 and L2Der are active with $\lambda = 5$. To validate this choice, we sweep $\lambda \in \{1, 2, 3, 4, 5\}$ on the RF model (Table 10).

At $\lambda = 1$, PDD_3 is highly sensitive (7–24 drifts), frequently retraining and achieving the lowest RMSE. At $\lambda = 5$ (our default), the method detects no drifts for 10 and 20 batches, indicating that natural PDP fluctuations remain below 5× the initial distance. The monotone relationship between λ and drift count confirms that the threshold is not arbitrary but provides a principled sensitivity–stability trade-off: practitioners monitoring high-stakes applications may prefer $\lambda = 1$, while noisier environments benefit from $\lambda = 2$ or higher.

Table 10
Threshold sensitivity: PDD_3 on `Friedman` (RF) with varying λ. Static-model RMSE (no retraining): ≈2.09.

λ	10 batches		20 batches		30 batches	
	RMSE	#drifts	RMSE	#drifts	RMSE	#drifts
1	2.005	7	1.942	17	1.943	24
2	2.091	0	2.020	4	2.035	6
3	2.091	0	2.098	0	2.075	1
4	2.091	0	2.098	0	2.094	0
5	2.091	0	2.098	0	2.094	0

5. Conclusions

In this paper, we conducted six experiments across multiple real-life and synthetic datasets to compare the effectiveness of our proposed method, PDD, and the most commonly used concept drift detection methods, including HDDM-A, HDDM-W, KSWIN, PH, DDM, and EDDM. We focused on analyzing model accuracy, the number of detected drifts, and understanding the variable contributions during drift occurrences.

PDD examines three metrics between two PDP curves: PDI, L2 (Euclidean distance), and L2Der (Euclidean distance between derivatives). By default, these three metrics are calculated from the PDPs derived from the train and test sets and are assigned as threshold values. For new incoming batches, these metrics from the test data are compared against the thresholds. In this study, we extended the original single-variable PDD to PDD_k, which monitors the top-k most important variables simultaneously; drift is triggered when any monitored variable exceeds all active thresholds. The special case $k = 1$ with fixed variable selection (PDD_C) recovers the original method, while PDD_1, PDD_2, and PDD_3 progressively increase monitoring coverage. Statistical tests also require adjusting their parameters, which can make them more or less sensitive depending on the type of data and drift. The same consideration applies to PDD_k: the threshold relaxation coefficient λ provides a principled sensitivity–stability trade-off, as demonstrated by the sweep on the `Friedman` dataset where increasing λ from 1 to 5 monotonically reduces drift counts from 24 to 0 at 30 batches.

The main purpose of this paper is not only to detect concept drift but also to better understand the relationship between the model and variables when drift occurs. While traditional statistical methods are generally used to detect concept drift based on whether the response variable can be correctly predicted, PDD can be used for both detecting concept drift and gaining a deeper understanding of the changes. Additionally, since PDP does not rely on whether the response variable is correctly predicted, it was able to detect concept drift even at high accuracy levels. This allows for detecting changes when the relationships in the data change, but no change is observed in the model performance metrics, such as accuracy, F1, etc., thereby enabling the model to be updated accordingly.

The PDD method, a primary focus of our analysis, demonstrated competitive performance in terms of both accuracy and drift detection consistency. It effectively balanced the detection of drifts while maintaining high accuracy across multiple datasets, particularly when compared to more sensitive methods like KSWIN and EDDM, which frequently detected a higher number of drifts, including potential false positives. PDD's conservative approach to drift detection proved beneficial in controlling unnecessary drift alarms, which is crucial for maintaining stability in real-world applications.

In the `SEA` and `Elec2` datasets, PDD showed stable performance by controlling the number of drift detections while achieving high accuracy rates. The experiments on the `Hyperplane` dataset highlighted PDD's sensitivity to batch size, where performance decreased beyond the 20-batch level, likely due to overfitting effects. This limitation was also observed in the `NOAA` and `Ozone` datasets, where overfitting in the RF model impacted the PDD's drift detection capability. However, in the `Friedman` dataset, PDD maintained a good balance between detecting significant drifts and minimizing false positives, providing consistent metric values across different models and batch sizes.

Comparatively, methods like KSWIN and EDDM exhibited higher sensitivity to concept drift, detecting a greater number of drifts across all experiments. While this high sensitivity is useful for capturing real changes in the data stream, it also leads to an increased risk of false positives, which may be undesirable in operational settings. On the other hand, PH and DDM provided moderate performance, striking a balance between drift detection and stability without generating excessive alarms.

An important observation across all datasets was the consistency in identifying key variables that contributed to model predictions and drift detection. For instance, attributes like `V2` in `SEA`, `nswprice` in `Elec2`, and `feature_4` in the `Friedman` dataset were repeatedly identified as the most influential variables. This stability in variable ranking explains why PDD_C and PDD_1 produce nearly identical results on several datasets: when the top variable does not change after retraining, dynamic variable selection offers no advantage over the fixed-variable strategy. Conversely, on datasets where variable importance shifts across batches, PDD_2 and PDD_3 detected additional drifts that PDD_C missed, demonstrating the benefit of multi-variable monitoring.

Finally, the computational overhead of PDD — its main practical limitation — was shown to be effectively mitigated through the *compress then explain* approach. Grid compression ($G = 10$) and subsample reduction ($N = 30$) together yielded a 4.9× speedup on the `Elec2` benchmark, bringing PDD_3's runtime below that of baseline methods while preserving accuracy within 0.2% of the default configuration.

For datasets exhibiting gradual drift, such as `Hyperplane`, PDD should be considered as part of a hybrid monitoring strategy: its profile-based approach provides interpretability that traditional accuracy-based methods lack, while a lightweight performance monitor, e.g., an accuracy-based trigger, can act as a safety net for detecting slow, cumulative degradation that remains below the per-batch PDP threshold.

6. Discussion

Besides the detection ability of our proposed method, it can provide explanations to understand the reason for predictive churn. After drift detection, re-training ML models can unpredictably change certain model predictions [40]. This may lead to unreliable conclusions about the model predictions. Fortunately, PDD can provide more detailed information about the relationship between the model and the data compared to other statistical methods in such situations. In this way, model users can understand the reason behind the drift. Similarly, users may request the deletion of their data regarding data privacy, and removing these relationships from any model trained on this data is a challenging problem. [41] developed a method to address this issue by partitioning and ordering the data. PDD can contribute to such problems in the literature.

The original PDD method relied on a single variable for drift monitoring, which could overlook drifts driven by secondary variables. The PDD_k extension addresses this limitation by simultaneously tracking the top-k most important variables and dynamically updating the variable ranking upon each drift detection. Our experiments show that PDD_2 and PDD_3 capture additional drift events that PDD_C misses — particularly in the DT and RF models on the `Elec2` and `Friedman` datasets — while the increased monitoring cost scales linearly with k.

A further practical concern was the computational cost of PDP evaluation. The compression benchmark on the `Elec2` dataset demonstrated that combining subsample reduction ($N = 30$) with grid compression ($G = 10$) reduces PDD_3's runtime from 12.3 s to 2.5 s — a 4.9× speedup — with negligible accuracy loss. Under these settings, PDD is faster than all six baseline methods on the same dataset, as the baselines trigger frequent retraining events that dominate their total

Table A.11
The number of observations in the batches.

Dataset	#batch					
	10		20		30	
	train	test	train	test	train	test
SEA	13 332	3334	8000	2000	5713	1429
Hyperplane	5 332	1334	3200	800	2285	572
NOAA	4 842	1211	2904	727	2075	519
Ozone	675	169	404	102	289	73
Elec2	12 083	3021	7249	1813	5178	1295
Friedman	5 332	1334	3200	800	2285	572

runtime. This confirms that the *compress then explain* approach [39] is a viable strategy for deploying PDD in latency-sensitive environments.

From a practical standpoint, practitioners should be aware that PDD's reliability depends on the calibration of the underlying model. A large divergence between the train-set and test-set PDPs during the initial baseline phase — quantified by the L2 metric — even in the absence of drift, is a strong indicator of overfitting, and suggests that the relaxation coefficient λ should be increased or the model should be regularized. In such scenarios, using the top-k monitoring strategy distributes the detection signal across multiple stable features, reducing sensitivity to any single overfitted variable.

The PDD method still has some limitations. For example, the calculation of derivatives can become challenging for categorical variables with a small number of levels, which may prevent PDD from calculating its metrics. PDD can also be more affected by small changes in sparse vectors because the three metrics calculated for PDD are averaged across all selected observation points. For example, the PDI metric measures whether the directions of derivatives are the same. For vectors with different derivative directions in 5 out of 100 observations, the PDI is calculated as 0.05, whereas for vectors with different directions in 1 out of 5 observations, the PDI increases to 0.20.

On the other hand, the PDD method can work on binary classification or regression tasks. When the response variable has more than two classes, PDD can be adapted for a specific level of the response variable, but this adaptation and tracking can be costly.

In conclusion, PDD offers advantages and disadvantages compared to traditional concept drift detection methods. The experiments demonstrated that PDD generally detected fewer but more consistent drifts while maintaining similar accuracy rates to other tests. Additionally, PDD allows for a more detailed examination of the relationship between explanatory variables and the response variable in the model, providing valuable insights into how these relationships change during concept drift.

Further research

A key practical concern for PDD is the computational cost of PDP evaluation. In this study, the *compress then explain* strategy [39] was adopted to address this bottleneck: as reported in Table 8, combining subsample reduction ($N = 30$) with grid compression ($G = 10$) achieved a 4.9× speedup overthe default PDD_3 configuration while maintaining accuracy within 0.2 percentage points. These results, consistent with the theoretical analysis in Table 1, confirm that distribution compression is an effective lever for deploying PDD in time-sensitive settings.

Building on this foundation, several directions remain open. First, extending PDD to natively handle multi-class classification — rather than resorting to one-versus-rest decomposition — would widen its applicability. Second, the threshold relaxation coefficient λ was empirically validated through a grid sweep (Table 10), yet an adaptive mechanism that tunes λ automatically based on the running variance of PDP distances or through a sequential hypothesis-testing framework would reduce the need for manual calibration. Third, our `Hyperplane` experiments exposed a limitation of per-batch thresholding: when each batch shift is small but cumulative drift is substantial, PDD may fail to trigger. Aggregating PDP distances over a sliding window or applying cumulative-sum statistics could improve sensitivity to such gradual concept drifts. Fourth, replacing full model retraining upon drift detection with incremental or online learning updates would lower the per-drift cost and make PDD more practical for high-velocity data streams. Finally, PDD's inherent explainability — identifying which variables cause a drift and how their functional relationships evolve — opens applications in predictive churn analysis and data privacy contexts [41], where understanding post-retraining behavioral changes is both a regulatory and operational necessity.

CRediT authorship contribution statement

Ugur Dar: Writing – original draft, Visualization, Software, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation. **Mustafa Cavus:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Ugur Dar reports financial support was provided by Eskisehir Technical University. Ugur Dar reports financial support was provided by The Scientific and Technological Research Council of Türkiye. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work on this paper is financially supported by the Scientific and Technological Research Council of Turkiye under the 2210C National MSc Scholarship Program in the Priority Fields in Science and Technology, grant no. 1649B022303919 and the Eskisehir Technical University Scientific Research Projects Commission under grant no. 22LÖT175.

Appendix. The description of the batches

See Tables A.11–A.13.

Data availability

The materials for reproducing the experiments performed, and the dataset are accessible in the following repository: https://github.com/ugurdar/xai-mlops.

Table A.12
The most important variables in the batches (top-k initial selection). Variables are listed in decreasing order of importance. Fewer than three variables are reported when the model uses fewer predictors (e.g., multicollinearity causes NA coefficients in LR, or the decision tree does not split on all variables).

Dataset	Model	#batch		
		10	20	30
SEA	RF	V2; V1; V3	V2; V1; V3	V2; V1; V3
	LR	V2; V1; V3	V2; V1; V3	V2; V1; V3
	DT	V1; V2; V3	V1; V2; V3	V1; V2; V3
Hyperplane	RF	feature_1; feature_2	feature_1; feature_2	feature_1; feature_2
	LR	feature_1; feature_2	feature_1; feature_2	feature_1; feature_2
	DT	feature_1; feature_2	feature_1; feature_2	feature_2; feature_1
NOAA	RF	attr2; attr4; attr3	attr4; attr2; attr3	attr2; attr4; attr1
	LR	attr8; attr2; attr1	attr4; attr2; attr8	attr2; attr8; attr4
	DT	attr2; attr1; attr8	attr2; attr1; attr8	attr2; attr1; attr7
Ozone	RF	V56; V69; V61	V61; V25; V33	V61; V3; V42
	LR	V25; V6; V70	V19; V45; V3	V19; V44; V4
	DT	V31; V33; V32	V36; V35; V37	V36; V37; V38
Elec2	RF	nswprice; nswdemand; vicprice	nswprice; nswdemand; vicprice	nswprice; nswdemand; vicprice
	LR	nswprice; nswdemand	nswprice; nswdemand	nswprice; nswdemand
	DT	nswprice; nswdemand	nswprice; nswdemand	nswprice; nswdemand
Friedman	RF	feature_4; feature_2; feature_1	feature_4; feature_2; feature_1	feature_4; feature_1; feature_2
	Linear	feature_4; feature_2; feature_1	feature_4; feature_2; feature_1	feature_4; feature_2; feature_1
	DT	feature_4; feature_2; feature_1	feature_4; feature_1; feature_2	feature_4; feature_2; feature_1

Table A.13
The accuracies of the models in the batches.

Dataset	Model	#batch					
		10		20		30	
		test	train	test	train	test	train
SEA	RF	0.8537	0.9991	0.8896	0.9989	0.9007	0.9991
	LR	0.8579	0.8966	0.8946	0.9032	0.9007	0.9057
	DT	0.8576	0.8801	0.8696	0.8850	0.8797	0.8850
Hyperplane	RF	0.6524	0.9992	0.6692	0.9984	0.4799	0.9978
	LR	0.6165	0.7320	0.6717	0.7122	0.5113	0.7580
	DT	0.7079	0.7406	0.7266	0.7094	0.4276	0.7575
NOAA	RF	0.7723	0.9996	0.8338	0.9997	0.8154	0.9995
	LR	0.7748	0.7999	0.8118	0.8041	0.8173	0.8014
	DT	0.7500	0.7881	0.7734	0.7996	0.7808	0.7990
Ozone	RF	0.9941	1.0000	0.9806	1.0000	1.0000	1.0000
	LR	0.9647	0.9526	0.8155	1.0000	0.9054	1.0000
	DT	0.9882	0.9511	0.8544	0.9554	0.9730	0.9308
Elec2	RF	0.8107	0.8424	0.8142	0.8367	0.8040	0.8418
	LR	0.8154	0.8200	0.8043	0.8081	0.8156	0.8188
	DT	0.7955	0.8264	0.8649	0.8238	0.8025	0.8316
Friedman	RF	1.6639	0.7431	1.8074	0.7709	1.7661	0.7949
	Linear	2.6432	2.5926	2.5591	2.5579	2.5958	2.5463
	DT	3.0254	2.9843	3.1574	2.9204	3.0400	2.9744

References

[1] D.A. Cieslak, N.V. Chawla, A framework for monitoring classifiers' performance: When and why failure occurs? Knowl. Inf. Syst. 18 (2009) 83–108.

[2] R. Alaiz-Rodríguez, N. Japkowicz, Assessing the impact of changing environments on classifier performance, in: Advances in Artificial Intelligence: 21st Conference of the Canadian Society for Computational Studies of Intelligence, Proceedings, Vol. 21, Springer, 2008, pp. 13–24.

[3] J.G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N.V. Chawla, F. Herrera, A unifying view on dataset shift in classification, Pattern Recognit. 45 (2012) 521–530.

[4] L. Korycki, B. Krawczyk, Adversarial concept drift detection under poisoning attacks for robust data stream mining, Mach. Learn. 112 (2023) 4013–4048.

[5] P. Biecek, Model development process, 2019, arXiv preprint arXiv:1907.04461.

[6] C. Mougan, D.S. Nielsen, Monitoring model deterioration with explainable uncertainty estimation via non-parametric bootstrap, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2023, pp. 15037–15045.

[7] F. Fumagalli, M. Muschalik, E. Hüllermeier, B. Hammer, Incremental permutation feature importance (ipfi): Towards online explanations on data streams, Mach. Learn. 112 (2023) 4863–4903.

[8] A. Garg, N. Shukla, L. Marla, S. Somanchi, Distribution shift in airline customer behavior during COVID-19, 2021, arXiv preprint arXiv:2111.14938.

[9] S. Kulinski, D.I. Inouye, Towards explaining distribution shifts, in: International Conference on Machine Learning, PMLR, 2023, pp. 17931–17952.

[10] H. Sheng, W. Xuanqi, Z. Chang, W. Jiacheng, D. Pingxia, W. Yuwei, AIGC video detection based on the fusion of spatial-frequency-optical flow multimodal features, J. Syst. Eng. Electron. (2026).

[11] J. Ahmad, S. Latif, I.U. Khan, M.S. Alshehri, M.S. Khan, N. Alasbali, W. Jiang, An interpretable deep learning framework for intrusion detection in the industrial internet of things, Internet Things 33 (2025) 101681.

[12] J. Demšar, Z. Bosnić, Detecting concept drift in data streams using model explanation, Expert Syst. Appl. 92 (2018) 546–559.

[13] C. Duckworth, F.P. Chmiel, D.K. Burns, Z.D. Zlatev, N.M. White, T.W.V. Daniels, M. Kiuber, M.J. Boniface, Using explainable machine learning to characterize data drift and detect emergent health risks for emergency department admissions during COVID-19, Sci. Rep. 11 (2021) 23017.

[14] K. Rahmani, R. Thapa, P. Tsou, S.C. Chetty, G. Barnes, C. Lam, C.F. Tso, Assessing the effects of data drift on the performance of machine learning models used in clinical sepsis prediction, Int. J. Med. Inform. 173 (2023) 104930.

[15] E. Strumbelj, I. Kononenko, M. Robnik-Sikonja, Explaining instance classifications with interactions of subsets of feature values, Data Knowl. Eng. 68 (2009) 886–904.

[16] S.M. Lundberg, S.I. Lee, A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems, Vol. 30, 2017, pp. 4765–4774.

[17] M. Muschalik, F. Fumagalli, B. Hammer, E. Hüllermeier, Agnostic explanation of model change based on feature importance, KI-Künstliche Intell. 36 (2022) 211–224.

[18] L. Breiman, Random forests, Mach. Learn. 45 (2001) 5–32.

[19] J.G. Mattos, T. Silva, H. Lopes, A.L. Bordignon, Interpretable concept drift, in: Pattern Recognition, Image Analysis, Computer Vision, and Applications, Springer, 2021, pp. 271–280.

[20] J. Moosbauer, J. Herbinger, G. Casalicchio, M. Lindauer, B. Bischl, Explaining hyperparameter optimization via partial dependence plots, in: Advances in Neural Information Processing Systems, 2021, pp. 2280–2291.

[21] A. Inglis, A. Parnell, C.B. Hurley, Visualizing variable importance and variable interaction effects in machine learning models, J. Comput. Graph. Statist. 31 (2022) 766–778.

[22] M. Cavus, J.N. van Rijn, P. Biecek, Beyond the single-best model: Rashomon partial dependence profile for trustworthy explanations in AutoML, in: International Conference on Discovery Science, Springer, 2025, pp. 445–459.

[23] M. Cavus, J.N. van Rijn, P. Biecek, Quantifying model uncertainty with AutoML and rashomon partial dependence profiles: Enabling trustworthy and human-centered XAI, Inf. Syst. Front. (2026) 1–19.

[24] I. Frías-Blanco, J.D. Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, Y. Caballero-Mota, Online and non-parametric drift detection methods based on Hoeffding's bounds, IEEE Trans. Knowl. Data Eng. 27 (2015) 810–823.

[25] C. Raab, M. Heusinger, F.M. Schleif, Reactive soft prototype computing for concept drift streams, 2020, arXiv preprint arXiv:2007.05432.

[26] R. Sebastiao, J.M. Fernandes, Supporting the Page–Hinkley test with empirical mode decomposition for change detection, in: International Symposium on Methodologies for Intelligent Systems, Springer, 2017, pp. 492–498.

[27] J. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: Advances in Artificial Intelligence, Springer, 2004, pp. 286–295.

[28] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, R. Morales-Bueno, Early drift detection method, in: Fourth International Workshop on Knowledge Discovery from Data Streams, 2006, pp. 77–86.
[29] K. Kobylinska, M. Krzyzinski, R. Machowicz, M. Adamek, P. Biecek, Exploration of the Rashomon set assists trustworthy explanations for medical data, IEEE J. Biomed. Health Inform. 28 (2024) 6454–6465.
[30] U. Dar, M. Cavus, A novel concept drift detection method based on the partial dependence profile disparity index, in: 7th International Researchers, Statisticians, and Young Statisticians Congress, 2023.
[31] U. Dar, M. Cavus, Explainable detection of data drift using partial dependence disparity index in real datasets, in: V. International Applied Statistics Congress, 2024.
[32] U. Dar, M. Cavus, datadriftR: An R package for data drift detection, J. Open Source Softw. 11 (119) (2026) 9481.
[33] W.N. Street, Y. Kim, A streaming ensemble algorithm (SEA) for large-scale classification, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2001, pp. 377–382.
[34] G. Hulten, L. Spencer, P. Domingos, Mining time-changing data streams, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2001, pp. 97–106.
[35] R. Elwell, R. Polikar, Incremental learning of concept drift in nonstationary environments, IEEE Trans. Neural Netw. 22 (2011) 1517–1531.
[36] K. Zhang, W. Fan, Forecasting skewed biased stochastic ozone days: Analyses, solutions, and beyond, Knowl. Inf. Syst. 14 (2008) 299–326.
[37] M. Harries, N.S. Wales, Splice-2 Comparative Evaluation: Electricity Pricing, Technical Report, University of New South Wales, 1999.
[38] Elena Ikonomovska, J. Gama, S. Džeroski, Learning model trees from evolving data streams, Data Min. Knowl. Discov. 23 (2011) 128–168.
[39] H. Baniecki, G. Casalicchio, B. Bischl, P. Biecek, Efficient and accurate explanation estimation with distribution compression, 2024, arXiv preprint arXiv:2406.18334.
[40] J. Watson-Daniels, F. du Pin Calmon, A. D'Amour, C. Long, D.C. Parkes, B. Ustun, Predictive churn with the set of good models, 2024, arXiv preprint arXiv:2402.07745.
[41] L. Bourtoule, V. Chandrasekaran, C.A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot, Machine unlearning, 2021, arXiv preprint arXiv:1912.03817.